

```
In [ ]: import os

data_path = os.path.curdir

import pandas as pd

data = pd.read_csv(os.path.join(data_path, "V8.csv")) #pandas объект dataframe
```

```
In [ ]: data.head() # первые 5 строк
```

```
Out[ ]:      Unnamed: 0.1  Unnamed: 0  cost_price  profit_markup  depth_of_tree  sales_commission  profit
0                0          1      3500.0           2           21           1000    -16500
1                1          2      3500.0           2           26           1000    -21500
2                2          3         NaN           2           23           1000    -18500
3                3          4         NaN           3           12           1000     -4000
4                4          5      3500.0           2           12           1000     -7500
```

```
In [ ]: data.shape # размерность набора данных
```

```
Out[ ]: (500, 7)
```

```
In [ ]: data.dtypes # типы атрибутов
```

```
Out[ ]:      0
      Unnamed: 0.1  int64
      Unnamed: 0    int64
      cost_price   float64
      profit_markup int64
      depth_of_tree int64
      sales_commission int64
      profit       int64
```

dtype: object

```
In [ ]: data.info() # число строк, количество ненулевых значений и типов атрибутов
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0.1          500 non-null   int64
1   Unnamed: 0            500 non-null   int64
2   cost_price            244 non-null   float64
3   profit_markup         500 non-null   int64
4   depth_of_tree         500 non-null   int64
5   sales_commission      500 non-null   int64
6   profit                500 non-null   int64
dtypes: float64(1), int64(6)
memory usage: 27.5 KB
```

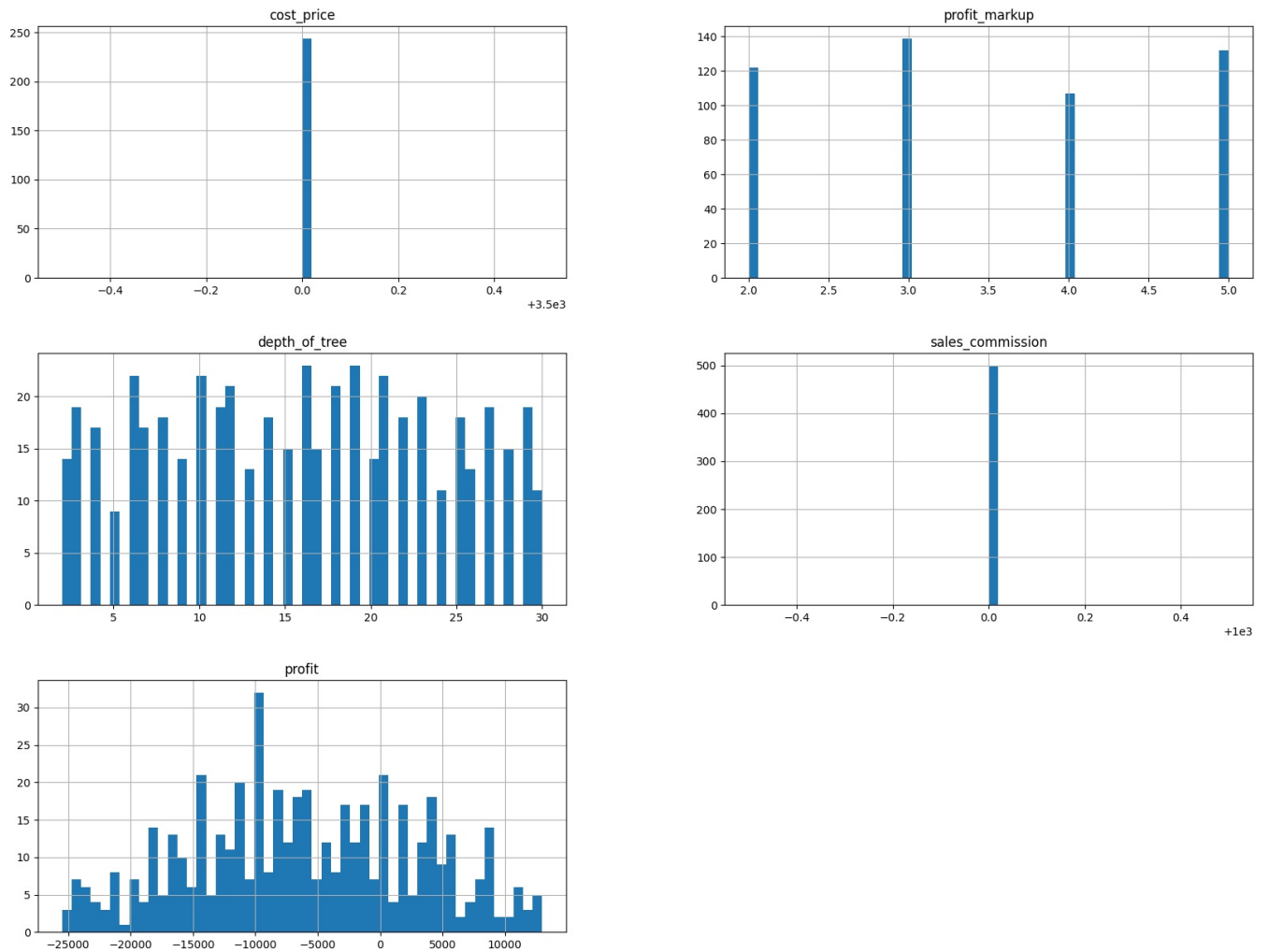
```
In [ ]: data.describe() # сводка по числовым атрибутам
```

```
Out[ ]:      Unnamed: 0.1  Unnamed: 0  cost_price  profit_markup  depth_of_tree  sales_commission  profit
count      500.000000    500.000000         244.0      500.000000    500.000000           500.0    500.000000
mean       249.500000    250.500000        3500.0        3.498000     15.896000           1000.0   -6153.000000
std        144.481833    144.481833           0.0        1.126292      8.095694           0.0    9080.528784
min          0.000000         1.000000        3500.0         2.000000      2.000000           1000.0  -25500.000000
25%        124.750000    125.750000        3500.0         3.000000      9.000000           1000.0  -12500.000000
50%        249.500000    250.500000        3500.0         3.000000     16.000000           1000.0   -6500.000000
75%        374.250000    375.250000        3500.0         5.000000     23.000000           1000.0     500.000000
max         499.000000    500.000000        3500.0         5.000000     30.000000           1000.0   13000.000000
```

```
In [ ]: # магическая команда для автоматического отображения
# визуализации при выполнении ячейки
# %matplotlib inline
```

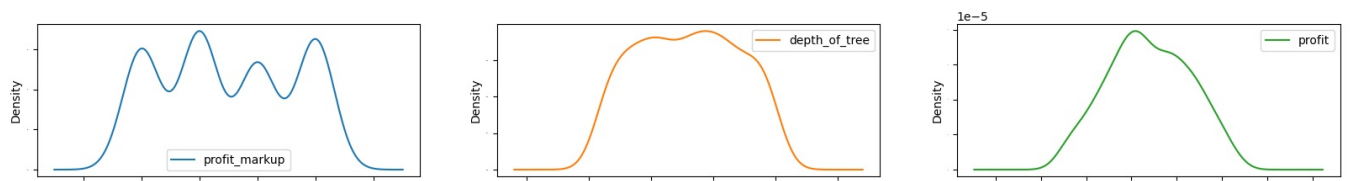
```
import matplotlib.pyplot as plt

data[['cost_price', 'profit_markup', 'depth_of_tree', 'sales_commission', 'profit']].hist(bins=50, figsize=(20,
plt.show())
```



Графики плотности

```
In [ ]: data[['profit_markup', 'depth_of_tree', 'profit']].\
plot(kind='density', subplots=True, layout=(3,3), sharex=False, legend=True,\
      fontsize=1, figsize=(20,8))
plt.show()
```



```
In [ ]: #: график корреляционной матрицы
```

```
import seaborn as sns

correlation = data[['profit_markup', 'depth_of_tree', 'profit']].corr()
plt.figure(figsize=(5,5))
plt.title('correlation matrix')
sns.heatmap(correlation, vmax=1, square=True, annot=True, cmap='cubehelix')
```

```
Out[ ]: <Axes: title={'center': 'correlation matrix'}>
```



```
In [ ]: # Стандартный коэф. корреляции
```

```
corr_matrix = data.corr()  
corr_matrix
```

```
Out[ ]:
```

	Unnamed: 0.1	Unnamed: 0	cost_price	profit_markup	depth_of_tree	sales_commission	profit	profit_markup_per_depth
Unnamed: 0.1	1.000000	1.000000	NaN	-0.013048	-0.011018	NaN	0.004159	
Unnamed: 0	1.000000	1.000000	NaN	-0.013048	-0.011018	NaN	0.004159	
cost_price	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
profit_markup	-0.013048	-0.013048	NaN	1.000000	-0.021562	NaN	0.453341	
depth_of_tree	-0.011018	-0.011018	NaN	-0.021562	1.000000	NaN	-0.900905	
sales_commission	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
profit	0.004159	0.004159	NaN	0.453341	-0.900905	NaN	1.000000	
profit_markup_per_depth	-0.005987	-0.005987	NaN	0.318411	-0.692209	NaN	0.755363	

```
In [ ]: corr_matrix["profit"].sort_values(ascending=False)
```

```
Out[ ]:
```

	profit
profit	1.000000
profit_markup_per_depth	0.755363
profit_markup	0.453341
Unnamed: 0.1	0.004159
Unnamed: 0	0.004159
depth_of_tree	-0.900905
cost_price	NaN
sales_commission	NaN

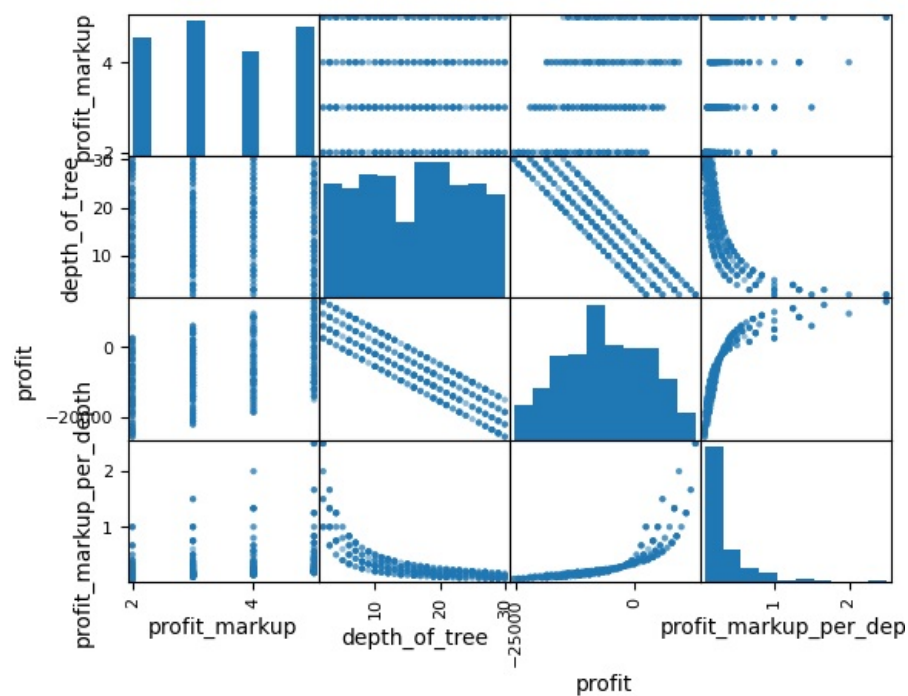
dtype: float64

```
In [ ]: # матрица диаграммы рассеяния
```

```
from pandas.plotting import scatter_matrix  
scatter_matrix(data[['profit_markup', 'depth_of_tree', 'profit', 'profit_markup_per_depth']])
```

```
Out[ ]:
```

```
array([[<Axes: xlabel='profit_markup', ylabel='profit_markup'>,  
       <Axes: xlabel='depth_of_tree', ylabel='profit_markup'>,  
       <Axes: xlabel='profit', ylabel='profit_markup'>,  
       <Axes: xlabel='profit_markup_per_depth', ylabel='profit_markup'>],  
       [[<Axes: xlabel='profit_markup', ylabel='depth_of_tree'>,  
         <Axes: xlabel='depth_of_tree', ylabel='depth_of_tree'>,  
         <Axes: xlabel='profit', ylabel='depth_of_tree'>,  
         <Axes: xlabel='profit_markup_per_depth', ylabel='depth_of_tree'>],  
       [[<Axes: xlabel='profit_markup', ylabel='profit'>,  
         <Axes: xlabel='depth_of_tree', ylabel='profit'>,  
         <Axes: xlabel='profit', ylabel='profit'>,  
         <Axes: xlabel='profit_markup_per_depth', ylabel='profit'>],  
       [[<Axes: xlabel='profit_markup', ylabel='profit_markup_per_depth'>,  
         <Axes: xlabel='depth_of_tree', ylabel='profit_markup_per_depth'>,  
         <Axes: xlabel='profit', ylabel='profit_markup_per_depth'>,  
         <Axes: xlabel='profit_markup_per_depth', ylabel='profit_markup_per_depth'>]],  
       dtype=object)
```



```
In [ ]: data_changed = data.drop(["Unnamed: 0", "Unnamed: 0.1", "cost_price", "sales_commission",], axis=1, inplace=False)
data_changed = data_changed[['profit_markup', 'depth_of_tree', 'profit']]
data_changed
```

```
Out[ ]:
```

	profit_markup	depth_of_tree	profit
0	2	21	-16500
1	2	26	-21500
2	2	23	-18500
3	3	12	-4000
4	2	12	-7500
...	...	...	...
495	5	12	3000
496	3	8	0
497	2	10	-5500
498	4	21	-9500
499	3	18	-10000

500 rows × 3 columns

```
In [ ]: from sklearn.feature_selection import SelectKBest, f_regression

data_X = data_changed.iloc[:, :3]
data_Y = data_changed['profit']

beastfeatures = SelectKBest(f_regression, k=3)
fit = beastfeatures.fit(data_X, data_Y)
dfscores = pd.DataFrame(fit.scores_)
dfcolumns = pd.DataFrame(data_X.columns)
```

```
featureScores = pd.concat([dfcolumns, dfscores],axis=1)
featureScores
```

```
Out[ ]:
```

	0	0
0	profit_markup	1.288237e+02
1	depth_of_tree	2.145721e+03
2	profit	-1.121396e+18

```
In [ ]: from sklearn.preprocessing import Normalizer
scaler = Normalizer().fit(data_changed)
Normalized_data_changed = pd.DataFrame(scaler.fit_transform(data_changed))
Normalized_data_changed
```

```
Out[ ]:
```

	0	1	2
0	0.000121	0.001273	-0.999999
1	0.000093	0.001209	-0.999999
2	0.000108	0.001243	-0.999999
3	0.000750	0.003000	-0.999995
4	0.000267	0.001600	-0.999999
...	...	...	...
495	0.001667	0.004000	0.999991
496	0.351123	0.936329	0.000000
497	0.000364	0.001818	-0.999998
498	0.000421	0.002211	-0.999997
499	0.000300	0.001800	-0.999998

500 rows × 3 columns

Разделение на обучающую, проверочную и тестовую выборки данных для **преобразованного построенного набора данных**

```
In [ ]: ndc_X = Normalized_data_changed.iloc[:, :3]
ndc_Y = Normalized_data_changed.iloc[:, -1]

from sklearn.model_selection import train_test_split
test_size = 0.2
val_size = 0.25
seed = 7

ndc_X_train, ndc_X_test, ndc_Y_train, ndc_Y_test =\
train_test_split(ndc_X, ndc_Y, test_size=test_size, random_state=seed)

ndc_X_train, ndc_X_val, ndc_Y_train, ndc_Y_val =\
train_test_split(ndc_X_train, ndc_Y_train, test_size = val_size, random_state=seed)

ndc_X_train
```

```
Out[ ]:
```

	0	1	2
167	0.001250	0.004750	-0.999988
380	0.000500	0.000500	1.000000
357	0.000625	0.002875	-0.999996
224	0.000833	0.001500	0.999999
102	0.000625	0.000875	0.999999
...	...	...	...
116	0.000211	0.001474	-0.999999
263	0.000121	0.001273	-0.999999
298	0.000176	0.001471	-0.999999
255	0.001667	0.006000	-0.999981
144	0.000108	0.001243	-0.999999

300 rows × 3 columns

Разделение на обучающую, проверочную и тестовую выборки данных для **построенного набора данных**

```
In [ ]: dc_X = data_changed.iloc[:, :3]
dc_Y = data_changed['profit']

dc_X_train, dc_X_test, dc_Y_train, dc_Y_test = \
train_test_split(dc_X, dc_Y, test_size=test_size, random_state=seed)

dc_X_train, dc_X_val, dc_Y_train, dc_Y_val = \
train_test_split(dc_X_train, dc_Y_train, test_size = val_size, random_state=seed)

dc_X_train
```

```
Out[ ]:      profit_markup  depth_of_tree  profit
167              5             19  -4000
380              5              5  10000
357              5             23  -8000
224              5              9   6000
102              5              7   8000
...             ...             ...     ...
116              2             14  -9500
263              2             21 -16500
298              3             25 -17000
255              5             18  -3000
144              2             23 -18500
```

300 rows × 3 columns

Разделение на обучающую, проверочную и тестовую выборки данных для **исходного набора данных**

```
In [ ]: data['cost_price'] = data['cost_price'].fillna(3500)

d_X = data.iloc[:, :6]
d_Y = data['profit']

d_X_train, d_X_test, d_Y_train, d_Y_test = \
train_test_split(d_X, d_Y, test_size=test_size, random_state=seed)

d_X_train, d_X_val, d_Y_train, d_Y_val = \
train_test_split(d_X_train, d_Y_train, test_size = val_size, random_state=seed)

d_X_train
```

```
Out[ ]:      Unnamed: 0.1  Unnamed: 0  cost_price  profit_markup  depth_of_tree  sales_commission
167              167            168    3500.0              5             19             1000
380              380            381    3500.0              5              5             1000
357              357            358    3500.0              5             23             1000
224              224            225    3500.0              5              9             1000
102              102            103    3500.0              5              7             1000
...             ...             ...     ...             ...             ...             ...
116              116            117    3500.0              2             14             1000
263              263            264    3500.0              2             21             1000
298              298            299    3500.0              3             25             1000
255              255            256    3500.0              5             18             1000
144              144            145    3500.0              2             23             1000
```

300 rows × 6 columns

Разделение на обучающую, проверочную и тестовую выборки данных для **преобразованного исходного набора данных**

```
In [ ]: scaler = Normalizer().fit(data)
Normalized_data = pd.DataFrame(scaler.fit_transform(data))
Normalized_data
```

Out[]:	0	1	2	3	4	5	6	7
0	0.000000	0.000059	0.207140	0.000118	0.001243	0.059183	-0.976519	0.000006
1	0.000046	0.000092	0.160506	0.000092	0.001192	0.045859	-0.985968	0.000004
2	0.000106	0.000159	0.185630	0.000106	0.001220	0.053037	-0.981187	0.000005
3	0.000555	0.000740	0.647148	0.000555	0.002219	0.184900	-0.739598	0.000046
4	0.000480	0.000600	0.419832	0.000240	0.001439	0.119952	-0.899639	0.000020
...	...	...	...	...	...	...	...	...
495	0.103800	0.104010	0.733941	0.001048	0.002516	0.209697	0.629092	0.000087
496	0.133795	0.134065	0.944117	0.000809	0.002158	0.269748	0.000000	0.000101
497	0.074930	0.075080	0.527674	0.000302	0.001508	0.150764	-0.829202	0.000030
498	0.048834	0.048932	0.343207	0.000392	0.002059	0.098059	-0.931563	0.000019
499	0.046787	0.046881	0.328166	0.000281	0.001688	0.093762	-0.937617	0.000016

500 rows × 8 columns

```
In [ ]: nd_X = Normalized_data.iloc[:, :6]
nd_Y = Normalized_data.iloc[:, -1]

nd_X_train, nd_X_test, nd_Y_train, nd_Y_test = \
train_test_split(nd_X, nd_Y, test_size=test_size, random_state=seed)

nd_X_train, nd_X_val, nd_Y_train, nd_Y_val = \
train_test_split(nd_X_train, nd_Y_train, test_size = val_size, random_state=seed)

nd_X_train
```

Out[]:	0	1	2	3	4	5
167	0.030849	0.031033	0.646526	0.000924	0.003510	0.184722
380	0.035662	0.035756	0.328469	0.000469	0.000469	0.093848
357	0.040551	0.040664	0.397557	0.000568	0.002613	0.113588
224	0.031886	0.032028	0.498219	0.000712	0.001281	0.142348
102	0.011604	0.011717	0.398162	0.000569	0.000796	0.113760
...	...	...	...	...	...	...
116	0.011401	0.011499	0.343986	0.000197	0.001376	0.098282
263	0.015561	0.015621	0.207090	0.000118	0.001243	0.059169
298	0.017136	0.017193	0.201260	0.000173	0.001438	0.057503
255	0.053902	0.054113	0.739825	0.001057	0.003805	0.211379
144	0.007637	0.007690	0.185619	0.000106	0.001220	0.053034

300 rows × 6 columns

```
In [ ]: from sklearn.metrics import mean_squared_error, r2_score
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Input, SimpleRNN
import numpy as np

def build_fully_connected_model(input_shape):
    model = Sequential()
    model.add(Input(shape=input_shape))
    model.add(Dense(256, activation='relu')) # Увеличиваем количество нейронов для большего потенциала обучени
    model.add(Dense(128, activation='relu')) # Увеличиваем количество нейронов для большего потенциала обучени
    model.add(Dense(64, activation='relu'))
    model.add(Dense(32, activation='relu'))
    model.add(Dense(16, activation='relu')) # Добавляем больше слоев для глубины
    model.add(Dense(1))
    model.compile(optimizer='adam', loss='mse')
    return model

def build_simple_rnn_model(input_shape):
    model = Sequential()
    model.add(Input(shape=input_shape))
    model.add(SimpleRNN(64, activation='tanh', return_sequences=True)) # Первый рекуррентный слой
    model.add(SimpleRNN(64, activation='tanh', return_sequences=True)) # Второй рекуррентный слой
    model.add(SimpleRNN(64, activation='tanh', return_sequences=True)) # 3 рекуррентный слой
    model.add(SimpleRNN(32, activation='tanh')) # Последний слой
    model.add(Dense(32, activation='relu'))
    model.add(Dense(16, activation='relu')) # Добавляем больше Dense-слоев
    model.add(Dense(1))
```



```
model.compile(optimizer='adam', loss='mse')
return model
```

Обучение с преобразованными нормализованными данными

```
In [ ]: # Определяем размер входных данных для полносвязной модели
input_shape_fc = (ndc_X_train.shape[1],)
# Создание моделей с правильной формой входных данных
ndc_fc_model = build_fully_connected_model(input_shape_fc)

# Для рекуррентной модели можно использовать ту же форму, но с добавлением размерности
input_shape_rnn = (ndc_X_train.shape[1], 1)

ndc_rnn_model = build_simple_rnn_model(input_shape_rnn)
```

```
In [ ]: # Преобразуем данные для рекуррентной модели
ndc_X_train_rnn = ndc_X_train.values.reshape((ndc_X_train.shape[0], ndc_X_train.shape[1], 1))
ndc_X_val_rnn = ndc_X_val.values.reshape((ndc_X_val.shape[0], ndc_X_val.shape[1], 1))

# Обучение рекуррентной модели
ndc_rnn_model.fit(ndc_X_train_rnn, ndc_Y_train, epochs=100, batch_size=32)
```

```
Epoch 1/100
10/10 ----- 6s 12ms/step - loss: 0.7802
Epoch 2/100
10/10 ----- 0s 10ms/step - loss: 0.1670
Epoch 3/100
10/10 ----- 0s 5ms/step - loss: 0.0059
Epoch 4/100
10/10 ----- 0s 5ms/step - loss: 0.0032
Epoch 5/100
10/10 ----- 0s 7ms/step - loss: 0.0020
Epoch 6/100
10/10 ----- 0s 8ms/step - loss: 6.4369e-04
Epoch 7/100
10/10 ----- 0s 5ms/step - loss: 2.4729e-04
Epoch 8/100
10/10 ----- 0s 6ms/step - loss: 1.1356e-04
Epoch 9/100
10/10 ----- 0s 5ms/step - loss: 4.5893e-05
Epoch 10/100
10/10 ----- 0s 6ms/step - loss: 1.7620e-05
Epoch 11/100
10/10 ----- 0s 6ms/step - loss: 9.9255e-06
Epoch 12/100
10/10 ----- 0s 6ms/step - loss: 1.0194e-05
Epoch 13/100
10/10 ----- 0s 6ms/step - loss: 5.3819e-06
Epoch 14/100
10/10 ----- 0s 8ms/step - loss: 1.0228e-05
Epoch 15/100
10/10 ----- 0s 6ms/step - loss: 5.9890e-06
Epoch 16/100
10/10 ----- 0s 6ms/step - loss: 1.0247e-05
Epoch 17/100
10/10 ----- 0s 6ms/step - loss: 4.0575e-06
Epoch 18/100
10/10 ----- 0s 7ms/step - loss: 3.1999e-06
Epoch 19/100
10/10 ----- 0s 5ms/step - loss: 6.7983e-06
Epoch 20/100
10/10 ----- 0s 6ms/step - loss: 6.3639e-06
Epoch 21/100
10/10 ----- 0s 6ms/step - loss: 7.2478e-06
Epoch 22/100
10/10 ----- 0s 8ms/step - loss: 9.8033e-06
Epoch 23/100
10/10 ----- 0s 6ms/step - loss: 1.6164e-05
Epoch 24/100
10/10 ----- 0s 6ms/step - loss: 8.6198e-06
Epoch 25/100
10/10 ----- 0s 6ms/step - loss: 9.6104e-06
Epoch 26/100
10/10 ----- 0s 6ms/step - loss: 8.9162e-06
Epoch 27/100
10/10 ----- 0s 6ms/step - loss: 1.6762e-05
Epoch 28/100
10/10 ----- 0s 7ms/step - loss: 1.9988e-05
Epoch 29/100
10/10 ----- 0s 6ms/step - loss: 1.3015e-05
Epoch 30/100
10/10 ----- 0s 8ms/step - loss: 1.0101e-05
Epoch 31/100
10/10 ----- 0s 8ms/step - loss: 9.7574e-06
Epoch 32/100
10/10 ----- 0s 6ms/step - loss: 1.2229e-05
Epoch 33/100
10/10 ----- 0s 6ms/step - loss: 7.0713e-05
```

Epoch 34/100		
10/10		0s 8ms/step - loss: 3.4031e-05
Epoch 35/100		
10/10		0s 6ms/step - loss: 4.7455e-05
Epoch 36/100		
10/10		0s 6ms/step - loss: 9.7771e-05
Epoch 37/100		
10/10		0s 6ms/step - loss: 6.9808e-04
Epoch 38/100		
10/10		0s 7ms/step - loss: 7.2697e-04
Epoch 39/100		
10/10		0s 8ms/step - loss: 2.0199e-04
Epoch 40/100		
10/10		0s 8ms/step - loss: 7.1326e-05
Epoch 41/100		
10/10		0s 6ms/step - loss: 2.2046e-05
Epoch 42/100		
10/10		0s 6ms/step - loss: 7.1771e-06
Epoch 43/100		
10/10		0s 7ms/step - loss: 7.7658e-06
Epoch 44/100		
10/10		0s 6ms/step - loss: 5.1056e-06
Epoch 45/100		
10/10		0s 7ms/step - loss: 7.0475e-06
Epoch 46/100		
10/10		0s 7ms/step - loss: 6.6043e-06
Epoch 47/100		
10/10		0s 9ms/step - loss: 2.3883e-06
Epoch 48/100		
10/10		0s 6ms/step - loss: 2.6518e-06
Epoch 49/100		
10/10		0s 6ms/step - loss: 3.0256e-06
Epoch 50/100		
10/10		0s 6ms/step - loss: 3.4042e-06
Epoch 51/100		
10/10		0s 6ms/step - loss: 5.4182e-06
Epoch 52/100		
10/10		0s 6ms/step - loss: 3.4165e-06
Epoch 53/100		
10/10		0s 6ms/step - loss: 6.7275e-06
Epoch 54/100		
10/10		0s 7ms/step - loss: 9.4926e-06
Epoch 55/100		
10/10		0s 8ms/step - loss: 4.4699e-06
Epoch 56/100		
10/10		0s 6ms/step - loss: 1.4881e-06
Epoch 57/100		
10/10		0s 6ms/step - loss: 1.8294e-06
Epoch 58/100		
10/10		0s 5ms/step - loss: 1.2334e-06
Epoch 59/100		
10/10		0s 6ms/step - loss: 3.5927e-06
Epoch 60/100		
10/10		0s 5ms/step - loss: 1.9168e-06
Epoch 61/100		
10/10		0s 6ms/step - loss: 1.9683e-06
Epoch 62/100		
10/10		0s 6ms/step - loss: 1.7357e-06
Epoch 63/100		
10/10		0s 7ms/step - loss: 9.8142e-07
Epoch 64/100		
10/10		0s 8ms/step - loss: 1.6488e-06
Epoch 65/100		
10/10		0s 8ms/step - loss: 1.2729e-06
Epoch 66/100		
10/10		0s 6ms/step - loss: 1.9621e-06
Epoch 67/100		
10/10		0s 5ms/step - loss: 1.5354e-06
Epoch 68/100		
10/10		0s 6ms/step - loss: 2.0758e-06
Epoch 69/100		
10/10		0s 7ms/step - loss: 4.0188e-06
Epoch 70/100		
10/10		0s 6ms/step - loss: 1.6985e-06
Epoch 71/100		
10/10		0s 6ms/step - loss: 1.2383e-06
Epoch 72/100		
10/10		0s 7ms/step - loss: 9.5104e-07
Epoch 73/100		
10/10		0s 8ms/step - loss: 1.0051e-06
Epoch 74/100		
10/10		0s 5ms/step - loss: 1.0581e-06
Epoch 75/100		
10/10		0s 6ms/step - loss: 1.7112e-06
Epoch 76/100		
10/10		0s 6ms/step - loss: 2.5352e-06
Epoch 77/100		
10/10		0s 6ms/step - loss: 5.7812e-06
Epoch 78/100		

```

10/10 ————— 0s 7ms/step - loss: 1.3830e-06
Epoch 79/100
10/10 ————— 0s 6ms/step - loss: 1.1415e-06
Epoch 80/100
10/10 ————— 0s 8ms/step - loss: 1.3329e-06
Epoch 81/100
10/10 ————— 0s 5ms/step - loss: 9.2782e-07
Epoch 82/100
10/10 ————— 0s 7ms/step - loss: 1.4354e-06
Epoch 83/100
10/10 ————— 0s 6ms/step - loss: 7.8599e-07
Epoch 84/100
10/10 ————— 0s 6ms/step - loss: 7.8432e-07
Epoch 85/100
10/10 ————— 0s 9ms/step - loss: 6.7214e-07
Epoch 86/100
10/10 ————— 0s 9ms/step - loss: 5.5752e-07
Epoch 87/100
10/10 ————— 0s 14ms/step - loss: 9.2762e-07
Epoch 88/100
10/10 ————— 0s 9ms/step - loss: 8.7284e-07
Epoch 89/100
10/10 ————— 0s 9ms/step - loss: 6.2116e-07
Epoch 90/100
10/10 ————— 0s 9ms/step - loss: 7.6657e-07
Epoch 91/100
10/10 ————— 0s 9ms/step - loss: 5.8307e-07
Epoch 92/100
10/10 ————— 0s 8ms/step - loss: 7.8463e-07
Epoch 93/100
10/10 ————— 0s 11ms/step - loss: 6.3123e-07
Epoch 94/100
10/10 ————— 0s 9ms/step - loss: 8.8318e-07
Epoch 95/100
10/10 ————— 0s 12ms/step - loss: 7.8116e-07
Epoch 96/100
10/10 ————— 0s 9ms/step - loss: 1.4205e-06
Epoch 97/100
10/10 ————— 0s 11ms/step - loss: 1.4475e-06
Epoch 98/100
10/10 ————— 0s 10ms/step - loss: 1.3369e-06
Epoch 99/100
10/10 ————— 0s 10ms/step - loss: 1.1621e-06
Epoch 100/100
10/10 ————— 0s 9ms/step - loss: 7.4745e-07
Out[ ]: <keras.src.callbacks.history.History at 0x7bd8e5fd34f0>

```

```

In [ ]: # Преобразуем данные для полносвязной модели
ndc_X_train_fc = ndc_X_train.values
ndc_X_val_fc = ndc_X_val.values

# Обучение рекуррентной модели
ndc_fc_model.fit(ndc_X_train_fc, ndc_Y_train, epochs=100, batch_size=32)

Epoch 1/100
10/10 ————— 2s 3ms/step - loss: 0.7313
Epoch 2/100
10/10 ————— 0s 3ms/step - loss: 0.2002
Epoch 3/100
10/10 ————— 0s 3ms/step - loss: 0.0681
Epoch 4/100
10/10 ————— 0s 2ms/step - loss: 0.0167
Epoch 5/100
10/10 ————— 0s 3ms/step - loss: 0.0047
Epoch 6/100
10/10 ————— 0s 3ms/step - loss: 0.0016
Epoch 7/100
10/10 ————— 0s 3ms/step - loss: 4.0947e-04
Epoch 8/100
10/10 ————— 0s 3ms/step - loss: 1.4572e-04
Epoch 9/100
10/10 ————— 0s 3ms/step - loss: 5.3940e-05
Epoch 10/100
10/10 ————— 0s 3ms/step - loss: 2.7947e-05
Epoch 11/100
10/10 ————— 0s 3ms/step - loss: 1.1567e-05
Epoch 12/100
10/10 ————— 0s 2ms/step - loss: 4.7143e-06
Epoch 13/100
10/10 ————— 0s 3ms/step - loss: 1.7609e-06
Epoch 14/100
10/10 ————— 0s 3ms/step - loss: 6.9495e-07
Epoch 15/100
10/10 ————— 0s 2ms/step - loss: 2.9787e-07
Epoch 16/100
10/10 ————— 0s 3ms/step - loss: 3.0991e-07
Epoch 17/100
10/10 ————— 0s 2ms/step - loss: 1.3049e-07

```

Epoch 18/100		
10/10		0s 3ms/step - loss: 1.7794e-07
Epoch 19/100		
10/10		0s 3ms/step - loss: 1.4786e-07
Epoch 20/100		
10/10		0s 2ms/step - loss: 2.1572e-07
Epoch 21/100		
10/10		0s 3ms/step - loss: 1.2701e-07
Epoch 22/100		
10/10		0s 2ms/step - loss: 1.3449e-07
Epoch 23/100		
10/10		0s 2ms/step - loss: 1.0681e-07
Epoch 24/100		
10/10		0s 3ms/step - loss: 9.8412e-08
Epoch 25/100		
10/10		0s 4ms/step - loss: 2.0107e-07
Epoch 26/100		
10/10		0s 4ms/step - loss: 1.5262e-07
Epoch 27/100		
10/10		0s 3ms/step - loss: 1.5651e-07
Epoch 28/100		
10/10		0s 2ms/step - loss: 1.3718e-07
Epoch 29/100		
10/10		0s 3ms/step - loss: 2.0948e-07
Epoch 30/100		
10/10		0s 3ms/step - loss: 2.2396e-07
Epoch 31/100		
10/10		0s 3ms/step - loss: 1.3692e-07
Epoch 32/100		
10/10		0s 3ms/step - loss: 1.9620e-07
Epoch 33/100		
10/10		0s 3ms/step - loss: 2.5374e-07
Epoch 34/100		
10/10		0s 4ms/step - loss: 1.3432e-07
Epoch 35/100		
10/10		0s 3ms/step - loss: 2.0089e-07
Epoch 36/100		
10/10		0s 3ms/step - loss: 2.0616e-07
Epoch 37/100		
10/10		0s 2ms/step - loss: 2.1040e-07
Epoch 38/100		
10/10		0s 3ms/step - loss: 1.6877e-07
Epoch 39/100		
10/10		0s 3ms/step - loss: 9.0827e-08
Epoch 40/100		
10/10		0s 3ms/step - loss: 9.8212e-08
Epoch 41/100		
10/10		0s 3ms/step - loss: 1.5880e-07
Epoch 42/100		
10/10		0s 3ms/step - loss: 2.2181e-07
Epoch 43/100		
10/10		0s 3ms/step - loss: 1.8751e-07
Epoch 44/100		
10/10		0s 2ms/step - loss: 1.0296e-07
Epoch 45/100		
10/10		0s 2ms/step - loss: 1.1630e-07
Epoch 46/100		
10/10		0s 3ms/step - loss: 9.0260e-08
Epoch 47/100		
10/10		0s 3ms/step - loss: 1.2967e-07
Epoch 48/100		
10/10		0s 3ms/step - loss: 1.0331e-07
Epoch 49/100		
10/10		0s 2ms/step - loss: 1.5878e-07
Epoch 50/100		
10/10		0s 2ms/step - loss: 1.2840e-07
Epoch 51/100		
10/10		0s 3ms/step - loss: 7.5062e-08
Epoch 52/100		
10/10		0s 3ms/step - loss: 1.1708e-07
Epoch 53/100		
10/10		0s 2ms/step - loss: 1.3022e-07
Epoch 54/100		
10/10		0s 4ms/step - loss: 1.2228e-07
Epoch 55/100		
10/10		0s 4ms/step - loss: 1.2092e-07
Epoch 56/100		
10/10		0s 3ms/step - loss: 1.4789e-07
Epoch 57/100		
10/10		0s 3ms/step - loss: 9.9091e-08
Epoch 58/100		
10/10		0s 3ms/step - loss: 1.5986e-07
Epoch 59/100		
10/10		0s 3ms/step - loss: 1.4224e-07
Epoch 60/100		
10/10		0s 3ms/step - loss: 1.0196e-07
Epoch 61/100		
10/10		0s 3ms/step - loss: 1.0169e-07
Epoch 62/100		

```

10/10 ————— 0s 2ms/step - loss: 1.1190e-07
Epoch 63/100
10/10 ————— 0s 3ms/step - loss: 1.4670e-07
Epoch 64/100
10/10 ————— 0s 4ms/step - loss: 7.2535e-08
Epoch 65/100
10/10 ————— 0s 3ms/step - loss: 1.5470e-07
Epoch 66/100
10/10 ————— 0s 2ms/step - loss: 1.7127e-07
Epoch 67/100
10/10 ————— 0s 2ms/step - loss: 7.0485e-08
Epoch 68/100
10/10 ————— 0s 3ms/step - loss: 1.3667e-07
Epoch 69/100
10/10 ————— 0s 3ms/step - loss: 1.1289e-07
Epoch 70/100
10/10 ————— 0s 4ms/step - loss: 9.0620e-08
Epoch 71/100
10/10 ————— 0s 4ms/step - loss: 1.0764e-07
Epoch 72/100
10/10 ————— 0s 3ms/step - loss: 1.1340e-07
Epoch 73/100
10/10 ————— 0s 3ms/step - loss: 1.2264e-07
Epoch 74/100
10/10 ————— 0s 3ms/step - loss: 1.1986e-07
Epoch 75/100
10/10 ————— 0s 3ms/step - loss: 8.2669e-08
Epoch 76/100
10/10 ————— 0s 3ms/step - loss: 1.1812e-07
Epoch 77/100
10/10 ————— 0s 3ms/step - loss: 7.9435e-08
Epoch 78/100
10/10 ————— 0s 3ms/step - loss: 6.6628e-08
Epoch 79/100
10/10 ————— 0s 4ms/step - loss: 9.4950e-08
Epoch 80/100
10/10 ————— 0s 3ms/step - loss: 5.5464e-08
Epoch 81/100
10/10 ————— 0s 3ms/step - loss: 5.8587e-08
Epoch 82/100
10/10 ————— 0s 2ms/step - loss: 1.5584e-07
Epoch 83/100
10/10 ————— 0s 3ms/step - loss: 1.2391e-07
Epoch 84/100
10/10 ————— 0s 4ms/step - loss: 5.1324e-08
Epoch 85/100
10/10 ————— 0s 4ms/step - loss: 9.9007e-08
Epoch 86/100
10/10 ————— 0s 3ms/step - loss: 7.4634e-08
Epoch 87/100
10/10 ————— 0s 4ms/step - loss: 6.0726e-08
Epoch 88/100
10/10 ————— 0s 3ms/step - loss: 4.7415e-08
Epoch 89/100
10/10 ————— 0s 3ms/step - loss: 1.3758e-07
Epoch 90/100
10/10 ————— 0s 2ms/step - loss: 9.5657e-08
Epoch 91/100
10/10 ————— 0s 3ms/step - loss: 4.0727e-08
Epoch 92/100
10/10 ————— 0s 3ms/step - loss: 1.0645e-07
Epoch 93/100
10/10 ————— 0s 3ms/step - loss: 4.4388e-08
Epoch 94/100
10/10 ————— 0s 3ms/step - loss: 5.1213e-08
Epoch 95/100
10/10 ————— 0s 3ms/step - loss: 7.1341e-08
Epoch 96/100
10/10 ————— 0s 3ms/step - loss: 6.0319e-08
Epoch 97/100
10/10 ————— 0s 3ms/step - loss: 8.2692e-08
Epoch 98/100
10/10 ————— 0s 3ms/step - loss: 1.1838e-07
Epoch 99/100
10/10 ————— 0s 4ms/step - loss: 6.6320e-08
Epoch 100/100
10/10 ————— 0s 5ms/step - loss: 3.4588e-08

```

```
Out[ ]: <keras.src.callbacks.history.History at 0x7bd8d53817b0>
```

Обучение с преобразованными не нормализованными данными

```

In [ ]: # Определяем размер входных данных для полносвязной модели
input_shape_fc = (dc_X_train.shape[1],)
# Создание моделей с правильной формой входных данных
dc_fc_model = build_fully_connected_model(input_shape_fc)

# Для рекуррентной модели можно использовать ту же форму, но с добавлением размерности

```

```
input_shape_rnn = (dc_X_train.shape[1], 1)
dc_rnn_model = build_simple_rnn_model(input_shape_rnn)
```

```
In [ ]: # Преобразуем данные для рекуррентной модели
dc_X_train_rnn = dc_X_train.values.reshape((dc_X_train.shape[0], dc_X_train.shape[1], 1))
dc_X_val_rnn = dc_X_val.values.reshape((dc_X_val.shape[0], dc_X_val.shape[1], 1))

# Обучение рекуррентной модели
dc_rnn_model.fit(dc_X_train_rnn, dc_Y_train, epochs=100, batch_size=32)
```

```
Epoch 1/100
10/10 ----- 6s 6ms/step - loss: 119244096.0000
Epoch 2/100
10/10 ----- 0s 6ms/step - loss: 115679664.0000
Epoch 3/100
10/10 ----- 0s 5ms/step - loss: 126259360.0000
Epoch 4/100
10/10 ----- 0s 5ms/step - loss: 109306992.0000
Epoch 5/100
10/10 ----- 0s 6ms/step - loss: 114899032.0000
Epoch 6/100
10/10 ----- 0s 6ms/step - loss: 120284944.0000
Epoch 7/100
10/10 ----- 0s 8ms/step - loss: 116222888.0000
Epoch 8/100
10/10 ----- 0s 8ms/step - loss: 115661536.0000
Epoch 9/100
10/10 ----- 0s 9ms/step - loss: 126890464.0000
Epoch 10/100
10/10 ----- 0s 6ms/step - loss: 119469384.0000
Epoch 11/100
10/10 ----- 0s 7ms/step - loss: 117694272.0000
Epoch 12/100
10/10 ----- 0s 6ms/step - loss: 118620048.0000
Epoch 13/100
10/10 ----- 0s 7ms/step - loss: 122047800.0000
Epoch 14/100
10/10 ----- 0s 7ms/step - loss: 124799536.0000
Epoch 15/100
10/10 ----- 0s 6ms/step - loss: 106636392.0000
Epoch 16/100
10/10 ----- 0s 9ms/step - loss: 116774728.0000
Epoch 17/100
10/10 ----- 0s 9ms/step - loss: 119988168.0000
Epoch 18/100
10/10 ----- 0s 6ms/step - loss: 11885728.0000
Epoch 19/100
10/10 ----- 0s 7ms/step - loss: 111255016.0000
Epoch 20/100
10/10 ----- 0s 6ms/step - loss: 104057720.0000
Epoch 21/100
10/10 ----- 0s 6ms/step - loss: 98318360.0000
Epoch 22/100
10/10 ----- 0s 6ms/step - loss: 107866608.0000
Epoch 23/100
10/10 ----- 0s 6ms/step - loss: 110979368.0000
Epoch 24/100
10/10 ----- 0s 8ms/step - loss: 106834640.0000
Epoch 25/100
10/10 ----- 0s 8ms/step - loss: 108604912.0000
Epoch 26/100
10/10 ----- 0s 6ms/step - loss: 118149232.0000
Epoch 27/100
10/10 ----- 0s 6ms/step - loss: 108333080.0000
Epoch 28/100
10/10 ----- 0s 7ms/step - loss: 111623008.0000
Epoch 29/100
10/10 ----- 0s 6ms/step - loss: 114507840.0000
Epoch 30/100
10/10 ----- 0s 5ms/step - loss: 102780488.0000
Epoch 31/100
10/10 ----- 0s 6ms/step - loss: 108314984.0000
Epoch 32/100
10/10 ----- 0s 8ms/step - loss: 109306864.0000
Epoch 33/100
10/10 ----- 0s 7ms/step - loss: 116306808.0000
Epoch 34/100
10/10 ----- 0s 6ms/step - loss: 110117656.0000
Epoch 35/100
10/10 ----- 0s 6ms/step - loss: 106458008.0000
Epoch 36/100
10/10 ----- 0s 6ms/step - loss: 99923960.0000
Epoch 37/100
10/10 ----- 0s 6ms/step - loss: 102230960.0000
Epoch 38/100
10/10 ----- 0s 6ms/step - loss: 110872888.0000
Epoch 39/100
10/10 ----- 0s 7ms/step - loss: 96463952.0000
```

Epoch 40/100		
10/10		0s 9ms/step - loss: 89273472.0000
Epoch 41/100		
10/10		0s 8ms/step - loss: 91539472.0000
Epoch 42/100		
10/10		0s 7ms/step - loss: 93731488.0000
Epoch 43/100		
10/10		0s 6ms/step - loss: 90053872.0000
Epoch 44/100		
10/10		0s 6ms/step - loss: 94004408.0000
Epoch 45/100		
10/10		0s 6ms/step - loss: 96126464.0000
Epoch 46/100		
10/10		0s 7ms/step - loss: 92684872.0000
Epoch 47/100		
10/10		0s 5ms/step - loss: 87745184.0000
Epoch 48/100		
10/10		0s 8ms/step - loss: 90190224.0000
Epoch 49/100		
10/10		0s 6ms/step - loss: 77308360.0000
Epoch 50/100		
10/10		0s 7ms/step - loss: 82159216.0000
Epoch 51/100		
10/10		0s 6ms/step - loss: 90937032.0000
Epoch 52/100		
10/10		0s 6ms/step - loss: 93886952.0000
Epoch 53/100		
10/10		0s 7ms/step - loss: 89707512.0000
Epoch 54/100		
10/10		0s 6ms/step - loss: 78130784.0000
Epoch 55/100		
10/10		0s 6ms/step - loss: 78930024.0000
Epoch 56/100		
10/10		0s 9ms/step - loss: 76095320.0000
Epoch 57/100		
10/10		0s 7ms/step - loss: 72038776.0000
Epoch 58/100		
10/10		0s 6ms/step - loss: 71081952.0000
Epoch 59/100		
10/10		0s 6ms/step - loss: 75777616.0000
Epoch 60/100		
10/10		0s 7ms/step - loss: 70931568.0000
Epoch 61/100		
10/10		0s 6ms/step - loss: 68790232.0000
Epoch 62/100		
10/10		0s 6ms/step - loss: 65579400.0000
Epoch 63/100		
10/10		0s 6ms/step - loss: 63353952.0000
Epoch 64/100		
10/10		0s 8ms/step - loss: 61652616.0000
Epoch 65/100		
10/10		0s 6ms/step - loss: 59181480.0000
Epoch 66/100		
10/10		0s 6ms/step - loss: 65822872.0000
Epoch 67/100		
10/10		0s 6ms/step - loss: 62235352.0000
Epoch 68/100		
10/10		0s 6ms/step - loss: 57240720.0000
Epoch 69/100		
10/10		0s 6ms/step - loss: 55882768.0000
Epoch 70/100		
10/10		0s 6ms/step - loss: 46862620.0000
Epoch 71/100		
10/10		0s 8ms/step - loss: 51794348.0000
Epoch 72/100		
10/10		0s 7ms/step - loss: 52946572.0000
Epoch 73/100		
10/10		0s 6ms/step - loss: 45973076.0000
Epoch 74/100		
10/10		0s 6ms/step - loss: 54919828.0000
Epoch 75/100		
10/10		0s 10ms/step - loss: 49865328.0000
Epoch 76/100		
10/10		0s 9ms/step - loss: 47678092.0000
Epoch 77/100		
10/10		0s 9ms/step - loss: 49924124.0000
Epoch 78/100		
10/10		0s 12ms/step - loss: 43118964.0000
Epoch 79/100		
10/10		0s 9ms/step - loss: 49081956.0000
Epoch 80/100		
10/10		0s 9ms/step - loss: 49004224.0000
Epoch 81/100		
10/10		0s 9ms/step - loss: 37578544.0000
Epoch 82/100		
10/10		0s 10ms/step - loss: 44335668.0000
Epoch 83/100		
10/10		0s 10ms/step - loss: 41902536.0000
Epoch 84/100		

```

10/10 ————— 0s 9ms/step - loss: 32639536.0000
Epoch 85/100
10/10 ————— 0s 11ms/step - loss: 31299464.0000
Epoch 86/100
10/10 ————— 0s 10ms/step - loss: 32797818.0000
Epoch 87/100
10/10 ————— 0s 10ms/step - loss: 31013674.0000
Epoch 88/100
10/10 ————— 0s 11ms/step - loss: 37202916.0000
Epoch 89/100
10/10 ————— 0s 14ms/step - loss: 30624956.0000
Epoch 90/100
10/10 ————— 0s 11ms/step - loss: 26521702.0000
Epoch 91/100
10/10 ————— 0s 12ms/step - loss: 28674004.0000
Epoch 92/100
10/10 ————— 0s 11ms/step - loss: 26969154.0000
Epoch 93/100
10/10 ————— 0s 6ms/step - loss: 26029194.0000
Epoch 94/100
10/10 ————— 0s 6ms/step - loss: 24078386.0000
Epoch 95/100
10/10 ————— 0s 9ms/step - loss: 24475686.0000
Epoch 96/100
10/10 ————— 0s 7ms/step - loss: 22715696.0000
Epoch 97/100
10/10 ————— 0s 6ms/step - loss: 21686934.0000
Epoch 98/100
10/10 ————— 0s 7ms/step - loss: 19725102.0000
Epoch 99/100
10/10 ————— 0s 7ms/step - loss: 20672362.0000
Epoch 100/100
10/10 ————— 0s 6ms/step - loss: 17397792.0000

```

Out[]: <keras.src.callbacks.history.History at 0x7bd8d6cd1090>

```

In [ ]: # Преобразуем данные для полносвязной модели
dc_X_train_fc = dc_X_train.values
dc_X_val_fc = dc_X_val.values

# Обучение рекуррентной модели
dc_fc_model.fit(dc_X_train_fc, dc_Y_train, epochs=100, batch_size=32)

```

```

Epoch 1/100
10/10 ————— 2s 3ms/step - loss: 76491456.0000
Epoch 2/100
10/10 ————— 0s 3ms/step - loss: 9290482.0000
Epoch 3/100
10/10 ————— 0s 3ms/step - loss: 3968411.0000
Epoch 4/100
10/10 ————— 0s 3ms/step - loss: 795521.9375
Epoch 5/100
10/10 ————— 0s 3ms/step - loss: 718069.6875
Epoch 6/100
10/10 ————— 0s 3ms/step - loss: 94103.1250
Epoch 7/100
10/10 ————— 0s 3ms/step - loss: 76792.6484
Epoch 8/100
10/10 ————— 0s 4ms/step - loss: 35586.0859
Epoch 9/100
10/10 ————— 0s 3ms/step - loss: 10341.2402
Epoch 10/100
10/10 ————— 0s 3ms/step - loss: 5730.0454
Epoch 11/100
10/10 ————— 0s 2ms/step - loss: 3958.3105
Epoch 12/100
10/10 ————— 0s 3ms/step - loss: 860.2377
Epoch 13/100
10/10 ————— 0s 3ms/step - loss: 352.2870
Epoch 14/100
10/10 ————— 0s 3ms/step - loss: 263.9751
Epoch 15/100
10/10 ————— 0s 3ms/step - loss: 154.5998
Epoch 16/100
10/10 ————— 0s 3ms/step - loss: 22.3945
Epoch 17/100
10/10 ————— 0s 3ms/step - loss: 9.2447
Epoch 18/100
10/10 ————— 0s 3ms/step - loss: 8.4413
Epoch 19/100
10/10 ————— 0s 3ms/step - loss: 6.8502
Epoch 20/100
10/10 ————— 0s 4ms/step - loss: 2.3892
Epoch 21/100
10/10 ————— 0s 3ms/step - loss: 1.1526
Epoch 22/100
10/10 ————— 0s 2ms/step - loss: 0.6976
Epoch 23/100
10/10 ————— 0s 4ms/step - loss: 0.6900

```


Epoch 24/100		
10/10		0s 4ms/step - loss: 0.4979
Epoch 25/100		
10/10		0s 3ms/step - loss: 0.4929
Epoch 26/100		
10/10		0s 3ms/step - loss: 0.4994
Epoch 27/100		
10/10		0s 2ms/step - loss: 0.3953
Epoch 28/100		
10/10		0s 2ms/step - loss: 0.4248
Epoch 29/100		
10/10		0s 2ms/step - loss: 0.3654
Epoch 30/100		
10/10		0s 3ms/step - loss: 0.3903
Epoch 31/100		
10/10		0s 3ms/step - loss: 0.4315
Epoch 32/100		
10/10		0s 3ms/step - loss: 0.3904
Epoch 33/100		
10/10		0s 3ms/step - loss: 0.3988
Epoch 34/100		
10/10		0s 3ms/step - loss: 0.4417
Epoch 35/100		
10/10		0s 2ms/step - loss: 0.4187
Epoch 36/100		
10/10		0s 3ms/step - loss: 0.3743
Epoch 37/100		
10/10		0s 5ms/step - loss: 0.3929
Epoch 38/100		
10/10		0s 4ms/step - loss: 0.4276
Epoch 39/100		
10/10		0s 3ms/step - loss: 0.4048
Epoch 40/100		
10/10		0s 2ms/step - loss: 0.3864
Epoch 41/100		
10/10		0s 3ms/step - loss: 0.4249
Epoch 42/100		
10/10		0s 3ms/step - loss: 0.4244
Epoch 43/100		
10/10		0s 3ms/step - loss: 0.4471
Epoch 44/100		
10/10		0s 3ms/step - loss: 0.5016
Epoch 45/100		
10/10		0s 3ms/step - loss: 0.4565
Epoch 46/100		
10/10		0s 3ms/step - loss: 0.3952
Epoch 47/100		
10/10		0s 3ms/step - loss: 0.4090
Epoch 48/100		
10/10		0s 3ms/step - loss: 0.4057
Epoch 49/100		
10/10		0s 3ms/step - loss: 0.4462
Epoch 50/100		
10/10		0s 3ms/step - loss: 0.3903
Epoch 51/100		
10/10		0s 4ms/step - loss: 0.4042
Epoch 52/100		
10/10		0s 3ms/step - loss: 0.4051
Epoch 53/100		
10/10		0s 3ms/step - loss: 0.5015
Epoch 54/100		
10/10		0s 3ms/step - loss: 0.3668
Epoch 55/100		
10/10		0s 3ms/step - loss: 0.3513
Epoch 56/100		
10/10		0s 3ms/step - loss: 0.3932
Epoch 57/100		
10/10		0s 3ms/step - loss: 0.4332
Epoch 58/100		
10/10		0s 3ms/step - loss: 0.3600
Epoch 59/100		
10/10		0s 3ms/step - loss: 0.3259
Epoch 60/100		
10/10		0s 3ms/step - loss: 0.3690
Epoch 61/100		
10/10		0s 4ms/step - loss: 0.3832
Epoch 62/100		
10/10		0s 3ms/step - loss: 0.3882
Epoch 63/100		
10/10		0s 3ms/step - loss: 0.3914
Epoch 64/100		
10/10		0s 4ms/step - loss: 0.4321
Epoch 65/100		
10/10		0s 3ms/step - loss: 0.4351
Epoch 66/100		
10/10		0s 3ms/step - loss: 0.5138
Epoch 67/100		
10/10		0s 2ms/step - loss: 0.4537
Epoch 68/100		

```

10/10 ————— 0s 3ms/step - loss: 0.4241
Epoch 69/100
10/10 ————— 0s 3ms/step - loss: 0.3530
Epoch 70/100
10/10 ————— 0s 3ms/step - loss: 0.4037
Epoch 71/100
10/10 ————— 0s 3ms/step - loss: 0.3601
Epoch 72/100
10/10 ————— 0s 3ms/step - loss: 0.4561
Epoch 73/100
10/10 ————— 0s 3ms/step - loss: 0.3819
Epoch 74/100
10/10 ————— 0s 3ms/step - loss: 0.3901
Epoch 75/100
10/10 ————— 0s 3ms/step - loss: 0.3732
Epoch 76/100
10/10 ————— 0s 3ms/step - loss: 0.3704
Epoch 77/100
10/10 ————— 0s 4ms/step - loss: 0.3244
Epoch 78/100
10/10 ————— 0s 3ms/step - loss: 0.3509
Epoch 79/100
10/10 ————— 0s 4ms/step - loss: 0.3781
Epoch 80/100
10/10 ————— 0s 3ms/step - loss: 0.3587
Epoch 81/100
10/10 ————— 0s 3ms/step - loss: 0.3774
Epoch 82/100
10/10 ————— 0s 2ms/step - loss: 0.3638
Epoch 83/100
10/10 ————— 0s 3ms/step - loss: 0.4132
Epoch 84/100
10/10 ————— 0s 2ms/step - loss: 0.3971
Epoch 85/100
10/10 ————— 0s 3ms/step - loss: 0.3787
Epoch 86/100
10/10 ————— 0s 3ms/step - loss: 0.3508
Epoch 87/100
10/10 ————— 0s 3ms/step - loss: 0.3924
Epoch 88/100
10/10 ————— 0s 3ms/step - loss: 0.3354
Epoch 89/100
10/10 ————— 0s 3ms/step - loss: 0.3181
Epoch 90/100
10/10 ————— 0s 4ms/step - loss: 0.3411
Epoch 91/100
10/10 ————— 0s 3ms/step - loss: 0.3967
Epoch 92/100
10/10 ————— 0s 4ms/step - loss: 0.4066
Epoch 93/100
10/10 ————— 0s 3ms/step - loss: 0.4467
Epoch 94/100
10/10 ————— 0s 3ms/step - loss: 0.5066
Epoch 95/100
10/10 ————— 0s 3ms/step - loss: 0.3904
Epoch 96/100
10/10 ————— 0s 3ms/step - loss: 0.3880
Epoch 97/100
10/10 ————— 0s 3ms/step - loss: 0.3999
Epoch 98/100
10/10 ————— 0s 3ms/step - loss: 0.4168
Epoch 99/100
10/10 ————— 0s 3ms/step - loss: 0.4058
Epoch 100/100
10/10 ————— 0s 3ms/step - loss: 0.4532

```

```
Out[ ]: <keras.src.callbacks.history.History at 0x7bd8d46e2680>
```

Обучение на исходном наборе данных

```

In [ ]: # Определяем размер входных данных для полносвязной модели
input_shape_fc = (d_X_train.shape[1],)
# Создание моделей с правильной формой входных данных
d_fc_model = build_fully_connected_model(input_shape_fc)

# Для рекуррентной модели можно использовать ту же форму, но с добавлением размерности
input_shape_rnn = (d_X_train.shape[1], 1)
d_rnn_model = build_simple_rnn_model(input_shape_rnn)

```

```

In [ ]: # Преобразуем данные для рекуррентной модели
d_X_train_rnn = d_X_train.values.reshape((d_X_train.shape[0], d_X_train.shape[1], 1))
d_X_val_rnn = d_X_val.values.reshape((d_X_val.shape[0], d_X_val.shape[1], 1))

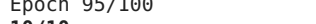
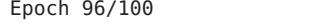
# Обучение рекуррентной модели
d_rnn_model.fit(d_X_train_rnn, d_Y_train, epochs=100, batch_size=32)

Epoch 1/100
10/10 ————— 6s 8ms/step - loss: 110357920.0000

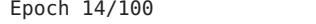
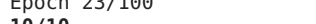
```

Epoch 2/100		
10/10		0s 10ms/step - loss: 124282544.0000
Epoch 3/100		
10/10		0s 8ms/step - loss: 116132824.0000
Epoch 4/100		
10/10		0s 8ms/step - loss: 121847784.0000
Epoch 5/100		
10/10		0s 8ms/step - loss: 115957784.0000
Epoch 6/100		
10/10		0s 8ms/step - loss: 121692296.0000
Epoch 7/100		
10/10		0s 8ms/step - loss: 120281016.0000
Epoch 8/100		
10/10		0s 8ms/step - loss: 117188576.0000
Epoch 9/100		
10/10		0s 11ms/step - loss: 119899072.0000
Epoch 10/100		
10/10		0s 11ms/step - loss: 116527344.0000
Epoch 11/100		
10/10		0s 8ms/step - loss: 107969488.0000
Epoch 12/100		
10/10		0s 9ms/step - loss: 114662240.0000
Epoch 13/100		
10/10		0s 8ms/step - loss: 117805832.0000
Epoch 14/100		
10/10		0s 8ms/step - loss: 119117800.0000
Epoch 15/100		
10/10		0s 8ms/step - loss: 114461464.0000
Epoch 16/100		
10/10		0s 9ms/step - loss: 110762632.0000
Epoch 17/100		
10/10		0s 10ms/step - loss: 112267760.0000
Epoch 18/100		
10/10		0s 9ms/step - loss: 121185000.0000
Epoch 19/100		
10/10		0s 8ms/step - loss: 110117104.0000
Epoch 20/100		
10/10		0s 8ms/step - loss: 116786568.0000
Epoch 21/100		
10/10		0s 8ms/step - loss: 115287384.0000
Epoch 22/100		
10/10		0s 9ms/step - loss: 121012632.0000
Epoch 23/100		
10/10		0s 8ms/step - loss: 102056920.0000
Epoch 24/100		
10/10		0s 11ms/step - loss: 109714352.0000
Epoch 25/100		
10/10		0s 9ms/step - loss: 108893192.0000
Epoch 26/100		
10/10		0s 9ms/step - loss: 111476712.0000
Epoch 27/100		
10/10		0s 9ms/step - loss: 115809120.0000
Epoch 28/100		
10/10		0s 8ms/step - loss: 107703088.0000
Epoch 29/100		
10/10		0s 8ms/step - loss: 98372040.0000
Epoch 30/100		
10/10		0s 9ms/step - loss: 98486888.0000
Epoch 31/100		
10/10		0s 11ms/step - loss: 108373736.0000
Epoch 32/100		
10/10		0s 8ms/step - loss: 107629232.0000
Epoch 33/100		
10/10		0s 10ms/step - loss: 111637728.0000
Epoch 34/100		
10/10		0s 8ms/step - loss: 105414224.0000
Epoch 35/100		
10/10		0s 8ms/step - loss: 105947592.0000
Epoch 36/100		
10/10		0s 8ms/step - loss: 98754800.0000
Epoch 37/100		
10/10		0s 9ms/step - loss: 99289832.0000
Epoch 38/100		
10/10		0s 8ms/step - loss: 97982568.0000
Epoch 39/100		
10/10		0s 11ms/step - loss: 93976032.0000
Epoch 40/100		
10/10		0s 10ms/step - loss: 92279576.0000
Epoch 41/100		
10/10		0s 8ms/step - loss: 99149336.0000
Epoch 42/100		
10/10		0s 9ms/step - loss: 96640016.0000
Epoch 43/100		
10/10		0s 8ms/step - loss: 82281496.0000
Epoch 44/100		
10/10		0s 9ms/step - loss: 95307104.0000
Epoch 45/100		
10/10		0s 8ms/step - loss: 88858584.0000
Epoch 46/100		

10/10	0s	10ms/step	-	loss: 86389384.0000
Epoch 47/100				
10/10	0s	10ms/step	-	loss: 85769176.0000
Epoch 48/100				
10/10	0s	8ms/step	-	loss: 91915376.0000
Epoch 49/100				
10/10	0s	10ms/step	-	loss: 86585136.0000
Epoch 50/100				
10/10	0s	8ms/step	-	loss: 74468024.0000
Epoch 51/100				
10/10	0s	10ms/step	-	loss: 75146800.0000
Epoch 52/100				
10/10	0s	8ms/step	-	loss: 76621984.0000
Epoch 53/100				
10/10	0s	8ms/step	-	loss: 80748728.0000
Epoch 54/100				
10/10	0s	11ms/step	-	loss: 66703924.0000
Epoch 55/100				
10/10	0s	9ms/step	-	loss: 66553440.0000
Epoch 56/100				
10/10	0s	10ms/step	-	loss: 72677168.0000
Epoch 57/100				
10/10	0s	8ms/step	-	loss: 64196360.0000
Epoch 58/100				
10/10	0s	14ms/step	-	loss: 60637156.0000
Epoch 59/100				
10/10	0s	13ms/step	-	loss: 63803880.0000
Epoch 60/100				
10/10	0s	14ms/step	-	loss: 60040936.0000
Epoch 61/100				
10/10	0s	14ms/step	-	loss: 56476016.0000
Epoch 62/100				
10/10	0s	12ms/step	-	loss: 62980216.0000
Epoch 63/100				
10/10	0s	17ms/step	-	loss: 58135900.0000
Epoch 64/100				
10/10	0s	13ms/step	-	loss: 55592268.0000
Epoch 65/100				
10/10	0s	14ms/step	-	loss: 49389456.0000
Epoch 66/100				
10/10	0s	16ms/step	-	loss: 50310540.0000
Epoch 67/100				
10/10	0s	13ms/step	-	loss: 51184312.0000
Epoch 68/100				
10/10	0s	9ms/step	-	loss: 49843500.0000
Epoch 69/100				
10/10	0s	8ms/step	-	loss: 44426008.0000
Epoch 70/100				
10/10	0s	8ms/step	-	loss: 40994288.0000
Epoch 71/100				
10/10	0s	9ms/step	-	loss: 36122260.0000
Epoch 72/100				
10/10	0s	9ms/step	-	loss: 37008416.0000
Epoch 73/100				
10/10	0s	8ms/step	-	loss: 38644808.0000
Epoch 74/100				
10/10	0s	8ms/step	-	loss: 43251316.0000
Epoch 75/100				
10/10	0s	10ms/step	-	loss: 41605368.0000
Epoch 76/100				
10/10	0s	9ms/step	-	loss: 33979448.0000
Epoch 77/100				
10/10	0s	9ms/step	-	loss: 34073736.0000
Epoch 78/100				
10/10	0s	9ms/step	-	loss: 29901078.0000
Epoch 79/100				
10/10	0s	11ms/step	-	loss: 29564202.0000
Epoch 80/100				
10/10	0s	9ms/step	-	loss: 27334488.0000
Epoch 81/100				
10/10	0s	8ms/step	-	loss: 35139912.0000
Epoch 82/100				
10/10	0s	9ms/step	-	loss: 23722144.0000
Epoch 83/100				
10/10	0s	9ms/step	-	loss: 28145996.0000
Epoch 84/100				
10/10	0s	9ms/step	-	loss: 27989392.0000
Epoch 85/100				
10/10	0s	8ms/step	-	loss: 25716122.0000
Epoch 86/100				
10/10	0s	11ms/step	-	loss: 21971800.0000
Epoch 87/100				
10/10	0s	8ms/step	-	loss: 21488528.0000
Epoch 88/100				
10/10	0s	8ms/step	-	loss: 18227332.0000
Epoch 89/100				
10/10	0s	9ms/step	-	loss: 20777972.0000
Epoch 90/100				
10/10	0s	10ms/step	-	loss: 18035980.0000

```
Epoch 91/100
10/10  0s 9ms/step - loss: 19965144.0000
Epoch 92/100
10/10  0s 8ms/step - loss: 16726318.0000
Epoch 93/100
10/10  0s 10ms/step - loss: 14300595.0000
Epoch 94/100
10/10  0s 9ms/step - loss: 17743044.0000
Epoch 95/100
10/10  0s 8ms/step - loss: 15452452.0000
Epoch 96/100
10/10  0s 9ms/step - loss: 14768613.0000
Epoch 97/100
10/10  0s 9ms/step - loss: 13600531.0000
Epoch 98/100
10/10  0s 9ms/step - loss: 13014597.0000
Epoch 99/100
10/10  0s 8ms/step - loss: 14348843.0000
Epoch 100/100
10/10  0s 9ms/step - loss: 12411677.0000
Out[ ]: <keras.src.callbacks.history.History at 0x7bd8d69e62c0>
```

```
In [ ]: # Преобразуем данные для полносвязной модели
d_X_train_fc = d_X_train.values
d_X_val_fc = d_X_val.values
# Обучение рекуррентной модели
d_fc_model.fit(d_X_train_fc, d_Y_train, epochs=100, batch_size=32)
```

```
Epoch 1/100
10/10  2s 4ms/step - loss: 114252088.0000
Epoch 2/100
10/10  0s 3ms/step - loss: 106860424.0000
Epoch 3/100
10/10  0s 3ms/step - loss: 88020568.0000
Epoch 4/100
10/10  0s 3ms/step - loss: 75489784.0000
Epoch 5/100
10/10  0s 2ms/step - loss: 83520792.0000
Epoch 6/100
10/10  0s 3ms/step - loss: 82004544.0000
Epoch 7/100
10/10  0s 4ms/step - loss: 84748016.0000
Epoch 8/100
10/10  0s 3ms/step - loss: 86651376.0000
Epoch 9/100
10/10  0s 2ms/step - loss: 72601608.0000
Epoch 10/100
10/10  0s 3ms/step - loss: 76120576.0000
Epoch 11/100
10/10  0s 3ms/step - loss: 71809976.0000
Epoch 12/100
10/10  0s 3ms/step - loss: 79547672.0000
Epoch 13/100
10/10  0s 2ms/step - loss: 73638392.0000
Epoch 14/100
10/10  0s 2ms/step - loss: 79433968.0000
Epoch 15/100
10/10  0s 2ms/step - loss: 78633056.0000
Epoch 16/100
10/10  0s 3ms/step - loss: 76282792.0000
Epoch 17/100
10/10  0s 2ms/step - loss: 78333120.0000
Epoch 18/100
10/10  0s 2ms/step - loss: 72772176.0000
Epoch 19/100
10/10  0s 3ms/step - loss: 79807272.0000
Epoch 20/100
10/10  0s 2ms/step - loss: 77734128.0000
Epoch 21/100
10/10  0s 2ms/step - loss: 84603688.0000
Epoch 22/100
10/10  0s 3ms/step - loss: 75537672.0000
Epoch 23/100
10/10  0s 4ms/step - loss: 76485856.0000
Epoch 24/100
10/10  0s 3ms/step - loss: 85524256.0000
Epoch 25/100
10/10  0s 3ms/step - loss: 71175888.0000
Epoch 26/100
10/10  0s 3ms/step - loss: 78724176.0000
Epoch 27/100
10/10  0s 3ms/step - loss: 80272288.0000
Epoch 28/100
10/10  0s 3ms/step - loss: 72040632.0000
Epoch 29/100
10/10  0s 3ms/step - loss: 77556208.0000
Epoch 30/100
10/10  0s 3ms/step - loss: 79725280.0000
```

Epoch 31/100		
10/10		0s 3ms/step - loss: 77412632.0000
Epoch 32/100		
10/10		0s 3ms/step - loss: 79745912.0000
Epoch 33/100		
10/10		0s 2ms/step - loss: 75887760.0000
Epoch 34/100		
10/10		0s 4ms/step - loss: 75478216.0000
Epoch 35/100		
10/10		0s 4ms/step - loss: 76720192.0000
Epoch 36/100		
10/10		0s 3ms/step - loss: 79012944.0000
Epoch 37/100		
10/10		0s 3ms/step - loss: 76178280.0000
Epoch 38/100		
10/10		0s 3ms/step - loss: 81580960.0000
Epoch 39/100		
10/10		0s 3ms/step - loss: 77892512.0000
Epoch 40/100		
10/10		0s 3ms/step - loss: 82394432.0000
Epoch 41/100		
10/10		0s 3ms/step - loss: 76797560.0000
Epoch 42/100		
10/10		0s 3ms/step - loss: 79312576.0000
Epoch 43/100		
10/10		0s 3ms/step - loss: 74815832.0000
Epoch 44/100		
10/10		0s 3ms/step - loss: 80448952.0000
Epoch 45/100		
10/10		0s 2ms/step - loss: 71439240.0000
Epoch 46/100		
10/10		0s 4ms/step - loss: 77953888.0000
Epoch 47/100		
10/10		0s 3ms/step - loss: 75423096.0000
Epoch 48/100		
10/10		0s 2ms/step - loss: 71866272.0000
Epoch 49/100		
10/10		0s 3ms/step - loss: 73794760.0000
Epoch 50/100		
10/10		0s 4ms/step - loss: 77444456.0000
Epoch 51/100		
10/10		0s 4ms/step - loss: 73654160.0000
Epoch 52/100		
10/10		0s 3ms/step - loss: 76673416.0000
Epoch 53/100		
10/10		0s 2ms/step - loss: 77611816.0000
Epoch 54/100		
10/10		0s 2ms/step - loss: 71752016.0000
Epoch 55/100		
10/10		0s 3ms/step - loss: 74806768.0000
Epoch 56/100		
10/10		0s 4ms/step - loss: 72822832.0000
Epoch 57/100		
10/10		0s 4ms/step - loss: 85877672.0000
Epoch 58/100		
10/10		0s 18ms/step - loss: 77417792.0000
Epoch 59/100		
10/10		0s 16ms/step - loss: 71084784.0000
Epoch 60/100		
10/10		1s 7ms/step - loss: 73411360.0000
Epoch 61/100		
10/10		0s 4ms/step - loss: 79300672.0000
Epoch 62/100		
10/10		0s 4ms/step - loss: 77015632.0000
Epoch 63/100		
10/10		0s 5ms/step - loss: 74227048.0000
Epoch 64/100		
10/10		0s 7ms/step - loss: 78519552.0000
Epoch 65/100		
10/10		0s 11ms/step - loss: 73622416.0000
Epoch 66/100		
10/10		1s 5ms/step - loss: 73551448.0000
Epoch 67/100		
10/10		0s 4ms/step - loss: 77613144.0000
Epoch 68/100		
10/10		0s 5ms/step - loss: 80182152.0000
Epoch 69/100		
10/10		0s 5ms/step - loss: 77040336.0000
Epoch 70/100		
10/10		0s 12ms/step - loss: 76991736.0000
Epoch 71/100		
10/10		0s 19ms/step - loss: 75209784.0000
Epoch 72/100		
10/10		0s 10ms/step - loss: 74095720.0000
Epoch 73/100		
10/10		0s 3ms/step - loss: 75375368.0000
Epoch 74/100		
10/10		0s 3ms/step - loss: 71540528.0000
Epoch 75/100		

```

10/10 ————— 0s 3ms/step - loss: 76882496.0000
Epoch 76/100
10/10 ————— 0s 3ms/step - loss: 66580600.0000
Epoch 77/100
10/10 ————— 0s 2ms/step - loss: 69937256.0000
Epoch 78/100
10/10 ————— 0s 3ms/step - loss: 71770064.0000
Epoch 79/100
10/10 ————— 0s 2ms/step - loss: 69531992.0000
Epoch 80/100
10/10 ————— 0s 3ms/step - loss: 66247384.0000
Epoch 81/100
10/10 ————— 0s 3ms/step - loss: 64226388.0000
Epoch 82/100
10/10 ————— 0s 3ms/step - loss: 62188224.0000
Epoch 83/100
10/10 ————— 0s 3ms/step - loss: 62664100.0000
Epoch 84/100
10/10 ————— 0s 5ms/step - loss: 54085524.0000
Epoch 85/100
10/10 ————— 0s 3ms/step - loss: 49044896.0000
Epoch 86/100
10/10 ————— 0s 3ms/step - loss: 41932456.0000
Epoch 87/100
10/10 ————— 0s 3ms/step - loss: 31648508.0000
Epoch 88/100
10/10 ————— 0s 3ms/step - loss: 22821226.0000
Epoch 89/100
10/10 ————— 0s 3ms/step - loss: 14789678.0000
Epoch 90/100
10/10 ————— 0s 3ms/step - loss: 13991269.0000
Epoch 91/100
10/10 ————— 0s 3ms/step - loss: 11618757.0000
Epoch 92/100
10/10 ————— 0s 3ms/step - loss: 28074048.0000
Epoch 93/100
10/10 ————— 0s 3ms/step - loss: 17186336.0000
Epoch 94/100
10/10 ————— 0s 3ms/step - loss: 12962533.0000
Epoch 95/100
10/10 ————— 0s 5ms/step - loss: 12809288.0000
Epoch 96/100
10/10 ————— 0s 3ms/step - loss: 11614140.0000
Epoch 97/100
10/10 ————— 0s 3ms/step - loss: 12763688.0000
Epoch 98/100
10/10 ————— 0s 3ms/step - loss: 12736218.0000
Epoch 99/100
10/10 ————— 0s 3ms/step - loss: 9825385.0000
Epoch 100/100
10/10 ————— 0s 3ms/step - loss: 9795920.0000
Out[ ]: <keras.src.callbacks.history.History at 0x7bd8d416f100>

```

Обучение на исходных нормализованных данных

```

In [ ]: # Определяем размер входных данных для полносвязной модели
input_shape_fc = (nd_X_train.shape[1],)
# Создание моделей с правильной формой входных данных
nd_fc_model = build_fully_connected_model(input_shape_fc)

# Для рекуррентной модели можно использовать ту же форму, но с добавлением размерности
input_shape_rnn = (nd_X_train.shape[1], 1)
nd_rnn_model = build_simple_rnn_model(input_shape_rnn)

```

```

In [ ]: # Преобразуем данные для рекуррентной модели
nd_X_train_rnn = nd_X_train.values.reshape((nd_X_train.shape[0], nd_X_train.shape[1], 1))
nd_X_val_rnn = nd_X_val.values.reshape((nd_X_val.shape[0], nd_X_val.shape[1], 1))

# Обучение рекуррентной модели
nd_rnn_model.fit(nd_X_train_rnn, nd_Y_train, epochs=100, batch_size=32)

```

```

Epoch 1/100
10/10 ————— 4s 8ms/step - loss: 0.0746
Epoch 2/100
10/10 ————— 0s 8ms/step - loss: 0.0038
Epoch 3/100
10/10 ————— 0s 8ms/step - loss: 4.9305e-04
Epoch 4/100
10/10 ————— 0s 9ms/step - loss: 2.9748e-04
Epoch 5/100
10/10 ————— 0s 8ms/step - loss: 1.0201e-04
Epoch 6/100
10/10 ————— 0s 8ms/step - loss: 6.1758e-05
Epoch 7/100
10/10 ————— 0s 9ms/step - loss: 2.2334e-05
Epoch 8/100
10/10 ————— 0s 11ms/step - loss: 1.9654e-05

```

Epoch 9/100		
10/10		0s 10ms/step - loss: 1.0002e-05
Epoch 10/100		
10/10		0s 8ms/step - loss: 8.4779e-06
Epoch 11/100		
10/10		0s 9ms/step - loss: 6.3430e-06
Epoch 12/100		
10/10		0s 9ms/step - loss: 5.1700e-06
Epoch 13/100		
10/10		0s 8ms/step - loss: 5.7433e-06
Epoch 14/100		
10/10		0s 9ms/step - loss: 4.9733e-06
Epoch 15/100		
10/10		0s 11ms/step - loss: 4.3916e-06
Epoch 16/100		
10/10		0s 9ms/step - loss: 4.0709e-06
Epoch 17/100		
10/10		0s 10ms/step - loss: 4.3851e-06
Epoch 18/100		
10/10		0s 8ms/step - loss: 4.3372e-06
Epoch 19/100		
10/10		0s 8ms/step - loss: 3.6957e-06
Epoch 20/100		
10/10		0s 9ms/step - loss: 3.4435e-06
Epoch 21/100		
10/10		0s 8ms/step - loss: 3.7118e-06
Epoch 22/100		
10/10		0s 10ms/step - loss: 3.6815e-06
Epoch 23/100		
10/10		0s 10ms/step - loss: 3.0245e-06
Epoch 24/100		
10/10		0s 8ms/step - loss: 4.2228e-06
Epoch 25/100		
10/10		0s 9ms/step - loss: 3.4652e-06
Epoch 26/100		
10/10		0s 13ms/step - loss: 3.6889e-06
Epoch 27/100		
10/10		0s 13ms/step - loss: 3.5924e-06
Epoch 28/100		
10/10		0s 13ms/step - loss: 3.8501e-06
Epoch 29/100		
10/10		0s 15ms/step - loss: 2.9483e-06
Epoch 30/100		
10/10		0s 12ms/step - loss: 2.8360e-06
Epoch 31/100		
10/10		0s 13ms/step - loss: 2.8508e-06
Epoch 32/100		
10/10		0s 14ms/step - loss: 3.0728e-06
Epoch 33/100		
10/10		0s 15ms/step - loss: 2.7618e-06
Epoch 34/100		
10/10		0s 14ms/step - loss: 3.2440e-06
Epoch 35/100		
10/10		0s 14ms/step - loss: 2.8209e-06
Epoch 36/100		
10/10		0s 14ms/step - loss: 2.4833e-06
Epoch 37/100		
10/10		0s 8ms/step - loss: 2.2800e-06
Epoch 38/100		
10/10		0s 9ms/step - loss: 1.9614e-06
Epoch 39/100		
10/10		0s 9ms/step - loss: 2.3251e-06
Epoch 40/100		
10/10		0s 9ms/step - loss: 3.0579e-06
Epoch 41/100		
10/10		0s 8ms/step - loss: 3.5948e-06
Epoch 42/100		
10/10		0s 11ms/step - loss: 3.1349e-06
Epoch 43/100		
10/10		0s 8ms/step - loss: 2.6844e-06
Epoch 44/100		
10/10		0s 9ms/step - loss: 2.1020e-06
Epoch 45/100		
10/10		0s 9ms/step - loss: 2.2768e-06
Epoch 46/100		
10/10		0s 9ms/step - loss: 1.9723e-06
Epoch 47/100		
10/10		0s 9ms/step - loss: 1.7437e-06
Epoch 48/100		
10/10		0s 8ms/step - loss: 1.5494e-06
Epoch 49/100		
10/10		0s 9ms/step - loss: 2.0688e-06
Epoch 50/100		
10/10		0s 10ms/step - loss: 1.7330e-06
Epoch 51/100		
10/10		0s 10ms/step - loss: 1.6438e-06
Epoch 52/100		
10/10		0s 8ms/step - loss: 2.4191e-06
Epoch 53/100		

10/10	0s	8ms/step	-	loss: 1.7032e-06
Epoch 54/100				
10/10	0s	10ms/step	-	loss: 1.5758e-06
Epoch 55/100				
10/10	0s	9ms/step	-	loss: 1.3101e-06
Epoch 56/100				
10/10	0s	9ms/step	-	loss: 1.5472e-06
Epoch 57/100				
10/10	0s	8ms/step	-	loss: 1.5989e-06
Epoch 58/100				
10/10	0s	10ms/step	-	loss: 1.1031e-06
Epoch 59/100				
10/10	0s	10ms/step	-	loss: 1.4332e-06
Epoch 60/100				
10/10	0s	9ms/step	-	loss: 1.6147e-06
Epoch 61/100				
10/10	0s	9ms/step	-	loss: 1.7857e-06
Epoch 62/100				
10/10	0s	10ms/step	-	loss: 1.1713e-06
Epoch 63/100				
10/10	0s	8ms/step	-	loss: 1.2502e-06
Epoch 64/100				
10/10	0s	8ms/step	-	loss: 1.4563e-06
Epoch 65/100				
10/10	0s	8ms/step	-	loss: 1.1015e-06
Epoch 66/100				
10/10	0s	11ms/step	-	loss: 1.2922e-06
Epoch 67/100				
10/10	0s	8ms/step	-	loss: 9.4609e-07
Epoch 68/100				
10/10	0s	9ms/step	-	loss: 1.3915e-06
Epoch 69/100				
10/10	0s	9ms/step	-	loss: 2.1257e-06
Epoch 70/100				
10/10	0s	10ms/step	-	loss: 2.1136e-06
Epoch 71/100				
10/10	0s	9ms/step	-	loss: 1.0043e-06
Epoch 72/100				
10/10	0s	9ms/step	-	loss: 1.1353e-06
Epoch 73/100				
10/10	0s	10ms/step	-	loss: 7.7699e-07
Epoch 74/100				
10/10	0s	10ms/step	-	loss: 7.4408e-07
Epoch 75/100				
10/10	0s	9ms/step	-	loss: 7.7335e-07
Epoch 76/100				
10/10	0s	8ms/step	-	loss: 7.9368e-07
Epoch 77/100				
10/10	0s	9ms/step	-	loss: 7.0495e-07
Epoch 78/100				
10/10	0s	10ms/step	-	loss: 7.6211e-07
Epoch 79/100				
10/10	0s	9ms/step	-	loss: 9.0597e-07
Epoch 80/100				
10/10	0s	8ms/step	-	loss: 7.8819e-07
Epoch 81/100				
10/10	0s	11ms/step	-	loss: 6.6542e-07
Epoch 82/100				
10/10	0s	10ms/step	-	loss: 8.4543e-07
Epoch 83/100				
10/10	0s	8ms/step	-	loss: 7.8447e-07
Epoch 84/100				
10/10	0s	9ms/step	-	loss: 7.3944e-07
Epoch 85/100				
10/10	0s	9ms/step	-	loss: 6.1000e-07
Epoch 86/100				
10/10	0s	11ms/step	-	loss: 4.9203e-07
Epoch 87/100				
10/10	0s	8ms/step	-	loss: 8.9680e-07
Epoch 88/100				
10/10	0s	9ms/step	-	loss: 8.7070e-07
Epoch 89/100				
10/10	0s	11ms/step	-	loss: 5.7364e-07
Epoch 90/100				
10/10	0s	9ms/step	-	loss: 5.4505e-07
Epoch 91/100				
10/10	0s	9ms/step	-	loss: 7.3898e-07
Epoch 92/100				
10/10	0s	9ms/step	-	loss: 7.9346e-07
Epoch 93/100				
10/10	0s	10ms/step	-	loss: 1.0767e-06
Epoch 94/100				
10/10	0s	9ms/step	-	loss: 7.9339e-07
Epoch 95/100				
10/10	0s	10ms/step	-	loss: 4.4451e-07
Epoch 96/100				
10/10	0s	9ms/step	-	loss: 4.6866e-07
Epoch 97/100				
10/10	0s	9ms/step	-	loss: 5.4372e-07

```
Epoch 98/100
10/10 ————— 0s 8ms/step - loss: 5.6403e-07
Epoch 99/100
10/10 ————— 0s 9ms/step - loss: 8.3355e-07
Epoch 100/100
10/10 ————— 0s 9ms/step - loss: 6.8284e-07
Out[ ]: <keras.src.callbacks.history.History at 0x7bd8d3dae200>
```

```
In [ ]: # Преобразуем данные для полносвязной модели
nd_X_train_fc = nd_X_train.values
nd_X_val_fc = nd_X_val.values

# Обучение рекуррентной модели
nd_fc_model.fit(nd_X_train_fc, nd_Y_train, epochs=100, batch_size=32)
```

```
Epoch 1/100
10/10 ————— 2s 4ms/step - loss: 3.2614e-04
Epoch 2/100
10/10 ————— 0s 5ms/step - loss: 9.8703e-06
Epoch 3/100
10/10 ————— 0s 4ms/step - loss: 2.5449e-06
Epoch 4/100
10/10 ————— 0s 5ms/step - loss: 1.1836e-06
Epoch 5/100
10/10 ————— 0s 4ms/step - loss: 4.1364e-07
Epoch 6/100
10/10 ————— 0s 4ms/step - loss: 1.5157e-07
Epoch 7/100
10/10 ————— 0s 4ms/step - loss: 5.5679e-08
Epoch 8/100
10/10 ————— 0s 4ms/step - loss: 3.9924e-08
Epoch 9/100
10/10 ————— 0s 5ms/step - loss: 1.7061e-08
Epoch 10/100
10/10 ————— 0s 5ms/step - loss: 6.7714e-09
Epoch 11/100
10/10 ————— 0s 4ms/step - loss: 4.0891e-09
Epoch 12/100
10/10 ————— 0s 4ms/step - loss: 4.0656e-09
Epoch 13/100
10/10 ————— 0s 4ms/step - loss: 3.3331e-09
Epoch 14/100
10/10 ————— 0s 4ms/step - loss: 3.7573e-09
Epoch 15/100
10/10 ————— 0s 4ms/step - loss: 4.1035e-09
Epoch 16/100
10/10 ————— 0s 4ms/step - loss: 3.6193e-09
Epoch 17/100
10/10 ————— 0s 7ms/step - loss: 3.1945e-09
Epoch 18/100
10/10 ————— 0s 5ms/step - loss: 3.9450e-09
Epoch 19/100
10/10 ————— 0s 5ms/step - loss: 3.3113e-09
Epoch 20/100
10/10 ————— 0s 5ms/step - loss: 2.4524e-09
Epoch 21/100
10/10 ————— 0s 2ms/step - loss: 3.8573e-09
Epoch 22/100
10/10 ————— 0s 2ms/step - loss: 3.6026e-09
Epoch 23/100
10/10 ————— 0s 2ms/step - loss: 4.9711e-09
Epoch 24/100
10/10 ————— 0s 3ms/step - loss: 2.9408e-09
Epoch 25/100
10/10 ————— 0s 2ms/step - loss: 4.1008e-09
Epoch 26/100
10/10 ————— 0s 3ms/step - loss: 2.6078e-09
Epoch 27/100
10/10 ————— 0s 3ms/step - loss: 2.3714e-09
Epoch 28/100
10/10 ————— 0s 3ms/step - loss: 2.4935e-09
Epoch 29/100
10/10 ————— 0s 3ms/step - loss: 2.8243e-09
Epoch 30/100
10/10 ————— 0s 3ms/step - loss: 2.3431e-09
Epoch 31/100
10/10 ————— 0s 3ms/step - loss: 3.4763e-09
Epoch 32/100
10/10 ————— 0s 2ms/step - loss: 3.4541e-09
Epoch 33/100
10/10 ————— 0s 3ms/step - loss: 2.2502e-09
Epoch 34/100
10/10 ————— 0s 3ms/step - loss: 2.0548e-09
Epoch 35/100
10/10 ————— 0s 2ms/step - loss: 2.0386e-09
Epoch 36/100
10/10 ————— 0s 3ms/step - loss: 2.1464e-09
Epoch 37/100
```

10/10	0s	2ms/step	-	loss: 2.1133e-09
Epoch 38/100				
10/10	0s	3ms/step	-	loss: 1.8958e-09
Epoch 39/100				
10/10	0s	3ms/step	-	loss: 2.7889e-09
Epoch 40/100				
10/10	0s	3ms/step	-	loss: 2.4621e-09
Epoch 41/100				
10/10	0s	3ms/step	-	loss: 2.2860e-09
Epoch 42/100				
10/10	0s	3ms/step	-	loss: 2.1335e-09
Epoch 43/100				
10/10	0s	3ms/step	-	loss: 4.3195e-09
Epoch 44/100				
10/10	0s	3ms/step	-	loss: 1.5294e-09
Epoch 45/100				
10/10	0s	3ms/step	-	loss: 2.5925e-09
Epoch 46/100				
10/10	0s	3ms/step	-	loss: 1.0510e-08
Epoch 47/100				
10/10	0s	4ms/step	-	loss: 1.2096e-08
Epoch 48/100				
10/10	0s	3ms/step	-	loss: 7.7722e-09
Epoch 49/100				
10/10	0s	3ms/step	-	loss: 9.3673e-09
Epoch 50/100				
10/10	0s	3ms/step	-	loss: 8.3996e-09
Epoch 51/100				
10/10	0s	3ms/step	-	loss: 3.7121e-09
Epoch 52/100				
10/10	0s	3ms/step	-	loss: 6.9109e-09
Epoch 53/100				
10/10	0s	4ms/step	-	loss: 2.8721e-09
Epoch 54/100				
10/10	0s	3ms/step	-	loss: 3.1730e-09
Epoch 55/100				
10/10	0s	3ms/step	-	loss: 4.7409e-09
Epoch 56/100				
10/10	0s	3ms/step	-	loss: 5.5461e-09
Epoch 57/100				
10/10	0s	3ms/step	-	loss: 4.4689e-09
Epoch 58/100				
10/10	0s	2ms/step	-	loss: 2.1537e-09
Epoch 59/100				
10/10	0s	2ms/step	-	loss: 3.7735e-09
Epoch 60/100				
10/10	0s	3ms/step	-	loss: 1.7784e-09
Epoch 61/100				
10/10	0s	4ms/step	-	loss: 1.2965e-09
Epoch 62/100				
10/10	0s	3ms/step	-	loss: 5.3727e-09
Epoch 63/100				
10/10	0s	4ms/step	-	loss: 2.0026e-08
Epoch 64/100				
10/10	0s	3ms/step	-	loss: 5.2201e-08
Epoch 65/100				
10/10	0s	3ms/step	-	loss: 1.1265e-07
Epoch 66/100				
10/10	0s	3ms/step	-	loss: 9.4886e-08
Epoch 67/100				
10/10	0s	3ms/step	-	loss: 1.5346e-07
Epoch 68/100				
10/10	0s	3ms/step	-	loss: 8.2710e-08
Epoch 69/100				
10/10	0s	3ms/step	-	loss: 5.2454e-09
Epoch 70/100				
10/10	0s	3ms/step	-	loss: 6.9639e-09
Epoch 71/100				
10/10	0s	3ms/step	-	loss: 2.3550e-09
Epoch 72/100				
10/10	0s	3ms/step	-	loss: 2.7860e-09
Epoch 73/100				
10/10	0s	3ms/step	-	loss: 2.6276e-09
Epoch 74/100				
10/10	0s	4ms/step	-	loss: 7.4001e-09
Epoch 75/100				
10/10	0s	3ms/step	-	loss: 1.7121e-08
Epoch 76/100				
10/10	0s	3ms/step	-	loss: 1.1732e-08
Epoch 77/100				
10/10	0s	3ms/step	-	loss: 6.1377e-09
Epoch 78/100				
10/10	0s	3ms/step	-	loss: 7.9306e-09
Epoch 79/100				
10/10	0s	3ms/step	-	loss: 1.3025e-08
Epoch 80/100				
10/10	0s	2ms/step	-	loss: 1.4587e-08
Epoch 81/100				
10/10	0s	3ms/step	-	loss: 1.1488e-08

```

Epoch 82/100
10/10  0s 3ms/step - loss: 4.8750e-09
Epoch 83/100
10/10  0s 3ms/step - loss: 4.6301e-08
Epoch 84/100
10/10  0s 3ms/step - loss: 3.4671e-08
Epoch 85/100
10/10  0s 3ms/step - loss: 8.4522e-08
Epoch 86/100
10/10  0s 3ms/step - loss: 4.7510e-07
Epoch 87/100
10/10  0s 3ms/step - loss: 6.1563e-07
Epoch 88/100
10/10  0s 3ms/step - loss: 9.2706e-07
Epoch 89/100
10/10  0s 3ms/step - loss: 2.3674e-06
Epoch 90/100
10/10  0s 4ms/step - loss: 3.8134e-07
Epoch 91/100
10/10  0s 2ms/step - loss: 2.0830e-07
Epoch 92/100
10/10  0s 3ms/step - loss: 2.5831e-07
Epoch 93/100
10/10  0s 3ms/step - loss: 1.9937e-07
Epoch 94/100
10/10  0s 4ms/step - loss: 9.3040e-08
Epoch 95/100
10/10  0s 3ms/step - loss: 2.2920e-08
Epoch 96/100
10/10  0s 3ms/step - loss: 8.3991e-09
Epoch 97/100
10/10  0s 3ms/step - loss: 5.5297e-09
Epoch 98/100
10/10  0s 3ms/step - loss: 5.3120e-09
Epoch 99/100
10/10  0s 3ms/step - loss: 5.0307e-09
Epoch 100/100
10/10  0s 3ms/step - loss: 4.6161e-09
Out[ ]: <keras.src.callbacks.history.History at 0x7bd8d180f430>

```

Сравнение моделей

```

In [ ]: def compare_fc_model(fc_model, x_train, x_val, y_train, y_val):
    Y_fc_predict_train = fc_model.predict(x_train)
    Y_fc_predict_val = fc_model.predict(x_val)
    rmse_fc_train = np.sqrt(mean_squared_error(y_train, Y_fc_predict_train))
    rmse_fc_val = np.sqrt(mean_squared_error(y_val, Y_fc_predict_val))
    r2_fc_train = r2_score(y_train, Y_fc_predict_train)
    r2_fc_val = r2_score(y_val, Y_fc_predict_val)
    print('Rmse for train values', rmse_fc_train,
          '\nRmse for validation values', rmse_fc_val,
          '\nR2 for train values', r2_fc_train,
          '\nR2 for validation values', r2_fc_val)

    print('!For ndc fc model!:')
    compare_fc_model(ndc_fc_model, ndc_X_train, ndc_X_val, ndc_Y_train, ndc_Y_val)

    print('!For dc fc model!:')
    compare_fc_model(dc_fc_model, dc_X_train, dc_X_val, dc_Y_train, dc_Y_val)

    print('!For d fc model!:')
    compare_fc_model(d_fc_model, d_X_train, d_X_val, d_Y_train, d_Y_val)

    print('!For nd fc model!:')
    compare_fc_model(nd_fc_model, nd_X_train, nd_X_val, nd_Y_train, nd_Y_val)

```

```

!For ndc fc model!:
10/10 ————— 0s 10ms/step
4/4 ————— 0s 2ms/step
Rmse for train values 0.00024656178789272867
Rmse for validation values 0.00025491563551996167
R2 for train values 0.9999999166883052
R2 for validation values 0.9999999269848548
!For dc fc model!:
10/10 ————— 0s 9ms/step
4/4 ————— 0s 3ms/step
Rmse for train values 0.7679085798111807
Rmse for validation values 0.8394982504625648
R2 for train values 1.0
R2 for validation values 1.0
!For d fc model!:
10/10 ————— 0s 9ms/step
4/4 ————— 0s 3ms/step
Rmse for train values 3108.653404462252
Rmse for validation values 3072.3914343125
R2 for train values 0.8767775297164917
R2 for validation values 0.879652738571167
!For nd fc model!:
10/10 ————— 0s 8ms/step
4/4 ————— 0s 3ms/step
Rmse for train values 8.831993501377347e-05
Rmse for validation values 9.023068260524126e-05
R2 for train values -2.3172787876167824
R2 for validation values -1.831048419056649

```

Можно сделать вывод, что ndc модель лучшая из fc. Она имеет наименьшие значения RMSE и наилучшие R² среди моделей. Хотя R² всё ещё невысокое, оно гораздо лучше, чем у других моделей.

```

In [ ]: def compare_rnn_model(rnn_model, x_train, x_val, y_train, y_val):
    Y_fc_predict_train = rnn_model.predict(x_train)
    Y_fc_predict_val = rnn_model.predict(x_val)
    rmse_fc_train = np.sqrt(mean_squared_error(y_train, Y_fc_predict_train))
    rmse_fc_val = np.sqrt(mean_squared_error(y_val, Y_fc_predict_val))
    r2_fc_train = r2_score(y_train, Y_fc_predict_train)
    r2_fc_val = r2_score(y_val, Y_fc_predict_val)
    print('Rmse for train values', rmse_fc_train,
          '\nRmse for validation values', rmse_fc_val,
          '\nR2 for train values', r2_fc_train,
          '\nR2 for validation values', r2_fc_val)

print('!For ndc rnn model!:')
compare_rnn_model(ndc_rnn_model, ndc_X_train, ndc_X_val, ndc_Y_train, ndc_Y_val)

print('!For dc rnn model!:')
compare_rnn_model(dc_rnn_model, dc_X_train, dc_X_val, dc_Y_train, dc_Y_val)

print('!For d rnn model!:')
compare_rnn_model(d_rnn_model, d_X_train, d_X_val, d_Y_train, d_Y_val)

print('!For nd rnn model!:')
compare_rnn_model(nd_rnn_model, nd_X_train, nd_X_val, nd_Y_train, nd_Y_val)

```

```

!For ndc rnn model!:
10/10 ————— 1s 53ms/step
4/4 ————— 0s 3ms/step
Rmse for train values 0.0007675103694597866
Rmse for validation values 0.0007357426542001988
R2 for train values 0.9999991927223457
R2 for validation values 0.9999993917643463
!For dc rnn model!:
10/10 ————— 1s 52ms/step
4/4 ————— 0s 4ms/step
Rmse for train values 4430.295159096952
Rmse for validation values 4039.896800849758
R2 for train values 0.7497289180755615
R2 for validation values 0.7919232249259949
!For d rnn model!:
10/10 ————— 1s 73ms/step
4/4 ————— 0s 7ms/step
Rmse for train values 3481.388405622996
Rmse for validation values 3031.493565645202
R2 for train values 0.8454567193984985
R2 for validation values 0.8828353881835938
!For nd rnn model!:
10/10 ————— 1s 81ms/step
4/4 ————— 0s 6ms/step
Rmse for train values 0.0006624756012961961
Rmse for validation values 0.0008205005525586295
R2 for train values -185.63980240287572
R2 for validation values -233.09738752843998

```

можно сделать вывод, что rnn ndc модель лучшая из rnn моделей. Сравнивая её с fc ndc моделью она тоже выигрывает.

```
In [ ]: !pip install scikeras
```

```
Collecting scikeras
  Downloading scikeras-0.13.0-py3-none-any.whl.metadata (3.1 kB)
Requirement already satisfied: keras>=3.2.0 in /usr/local/lib/python3.10/dist-packages (from scikeras) (3.5.0)
Requirement already satisfied: scikit-learn>=1.4.2 in /usr/local/lib/python3.10/dist-packages (from scikeras) (1.5.2)
Requirement already satisfied: absl-py in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (1.4.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (1.26.4)
Requirement already satisfied: rich in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (0.0.8)
Requirement already satisfied: h5py in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (3.12.1)
Requirement already satisfied: optree in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (0.13.1)
Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (0.4.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (24.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=1.4.2->scikeras) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=1.4.2->scikeras) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=1.4.2->scikeras) (3.5.0)
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.10/dist-packages (from optree->keras>=3.2.0->scikeras) (4.12.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.10/dist-packages (from rich->keras>=3.2.0->scikeras) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.10/dist-packages (from rich->keras>=3.2.0->scikeras) (2.18.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.10/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.2.0->scikeras) (0.1.2)
Downloading scikeras-0.13.0-py3-none-any.whl (26 kB)
Installing collected packages: scikeras
Successfully installed scikeras-0.13.0
```

```
In [ ]: from sklearn.model_selection import GridSearchCV
from tensorflow.keras.optimizers import Adam

from scikeras.wrappers import KerasRegressor

# Функция для построения модели
def build_fully_connected_model(input_shape, units_dense=128, learning_rate=0.001):
    model = Sequential()
    model.add(Input(shape=input_shape))
    model.add(Dense(units_dense, activation='relu'))
    model.add(Dense(units_dense // 2, activation='relu'))
    model.add(Dense(units_dense // 4, activation='relu'))
    model.add(Dense(units_dense // 8, activation='relu'))
    model.add(Dense(1))
    model.compile(optimizer=Adam(learning_rate=learning_rate), loss='mse', metrics=['mae'])
    return model

# Обертка для GridSearchCV
model = KerasRegressor(
    model=build_fully_connected_model,
    input_shape=(ndc_X_train_fc.shape[1],), # Размер входных данных для fully-connected модели
    verbose=0 # Убираем вывод Keras для чистоты
)

# Гиперпараметры для подбора
param_grid = {
    "model__units_dense": [64, 128, 256], # Количество нейронов в Dense-слоях
    "batch_size": [16, 32, 64], # Размер батча
    "epochs": [100], # Количество эпох
    "model__learning_rate": [0.001, 0.01] # Скорость обучения
}

# Инициализация GridSearchCV
grid = GridSearchCV(estimator=model, param_grid=param_grid, cv=3, scoring='neg_mean_squared_error', verbose=1)

X_combined_fc = np.concatenate((ndc_X_train_fc, ndc_X_val_fc), axis=0)
Y_combined_fc = np.concatenate((ndc_Y_train, ndc_Y_val), axis=0)

# Запуск подбора гиперпараметров
grid_result = grid.fit(X_combined_fc, Y_combined_fc)
```

Fitting 3 folds for each of 18 candidates, totalling 54 fits

WARNING:tensorflow:5 out of the last 15 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7bd8c95497e0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

WARNING:tensorflow:5 out of the last 11 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7bd8c95497e0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

/usr/local/lib/python3.10/dist-packages/numpy/ma/core.py:2820: RuntimeWarning: invalid value encountered in cast
_data = np.array(data, dtype=dtype, copy=copy,

подбор для рекуррентной

In []:

```
In [ ]: # Получение лучшей модели
best_model = grid.best_estimator_

# Получение лучших гиперпараметров
best_params = grid.best_params_

# Получение лучшего результата (оценки)
best_score = grid.best_score_

# Вывод результатов
print("Лучшая модель:", best_model)
print("Лучшие гиперпараметры:", best_params)
print("Лучший результат (neg_mean_squared_error):", best_score)
```

```
Лучшая модель: KerasRegressor(
  model=<function build_fully_connected_model at 0x7bd8d2b0c790>
  build_fn=None
  warm_start=False
  random_state=None
  optimizer=rmsprop
  loss=None
  metrics=None
  batch_size=64
  validation_batch_size=None
  verbose=0
  callbacks=None
  validation_split=0.0
  shuffle=True
  run_eagerly=False
  epochs=100
  input_shape=(3,)
  model__learning_rate=0.001
  model__units_dense=256
)
```

Лучшие гиперпараметры: {'batch_size': 64, 'epochs': 100, 'model__learning_rate': 0.001, 'model__units_dense': 256}

Лучший результат (neg_mean_squared_error): -1.6160103892004284e-07

```
In [ ]: ndc_X_test_rnn = ndc_X_test.values.reshape((ndc_X_test.shape[0], ndc_X_test.shape[1], 1))
ndc_X_test_fc = ndc_X_test.values
Y_test_predict = best_model.predict(ndc_X_test_fc)

# Оценка RMSE
rmse_test = np.sqrt(mean_squared_error(ndc_Y_test, Y_test_predict))

# Оценка R²
r2_test = r2_score(ndc_Y_test, Y_test_predict)

print('RMSE на тестовом наборе:', rmse_test)
print('R² на тестовом наборе:', r2_test)
```

RMSE на тестовом наборе: 5.8108204396628935e-05

R² на тестовом наборе: 0.9999999953738505

In []: